THE HONG KONG UNIVERSITY OF SCIENCE AND TECHNOLOGY (GUANGZHOU)

DSAA2011 Machine Learning

L4 Supervised Learning: Regression and Classification III

Dr. Zixin Zhong

Data Science and Analytics Thrust
Information Hub
Hong Kong University of Science and Technology (Guangzhou)

February 26, 2026

Course feedback

♡ Please scan the QR code and share your feedback.
No worries as it is anonymous. We sincerely appreciate your help.



Syllabus

Lec #	Topic	Lecturer
1	Introduction + course Info	Zixin
2-4	**Supervised learning: regression and classification**	Zixin
5	SVM (supervised learning) + Feature engineering	Zixin
6	Model evaluation and choice + Feature selection	Zixin
7	Ensemble learning (supervised learning)	Weikai
	Midterm (28 March, 1:30 - 3:00 PM): mark it on calendar!	
8-9	Unsupervised learning: clustering	Weikai
10	Dimension reduction	Weikai
11	Markov and graphical models	Weikai
12	Active learning + Reinforcement learning	Weikai
13	Project presentation	Both
	Project report submission (TBD)	
	Final exam (TBD)	



Outline

1. Logistic regression

2. Support vector machines (SVM)



Recap of Lecture 3

- Ridge regression
- Gradient descent



Ridge regression (linear form)

- Ridge regression in primal form (when $m > d$)
  - **Learning/Training**:
$$\mathbf{w}^* = (\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I}_{d+1})^{-1} \mathbf{X}^\top \mathbf{y}$$
  - **Prediction/Testing**: Given a new sample $\mathbf{x}_{\text{new}}$
$$\hat{y}_{\text{new}} = \begin{bmatrix} 1 \\ \mathbf{x}_{\text{new}} \end{bmatrix}^\top \mathbf{w}^*$$
    where $\mathbf{p}_{\text{new}}$ is the polynomial vector associated to $\mathbf{x}_{\text{new}}$.

- Ridge regression in dual form (when $m < d' = \binom{p+d}{p}$)
  - **Learning/Training**:
$$\mathbf{w}^* = \mathbf{X}^\top (\mathbf{X}\mathbf{X}^\top + \lambda \mathbf{I}_m)^{-1} \mathbf{y}$$
  - **Prediction/Testing**: Given a new sample $\mathbf{x}_{\text{new}}$
$$\hat{y}_{\text{new}} = \begin{bmatrix} 1 \\ \mathbf{x}_{\text{new}} \end{bmatrix}^\top \mathbf{w}^*$$



Ridge regression (linear form)

- **Fact 1:** For every $\lambda > 0$, $\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I}_{d+1}$ is invertible.
  - Hint: $\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I}_{d+1}$ is positive-definite.

- **Fact 2:** For every $\lambda > 0$,

$$(\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I}_{d+1})^{-1} \mathbf{X}^\top \mathbf{y} = \mathbf{X}^\top (\mathbf{X}\mathbf{X}^\top + \lambda \mathbf{I}_m)^{-1} \mathbf{y}. \tag{P-D}$$

  - Hint: To show (P-D), we use the Woodbury formula

$$(\mathbf{I} + \mathbf{U}\mathbf{V})^{-1} = \mathbf{I} - \mathbf{U}(\mathbf{I} + \mathbf{V}\mathbf{U})^{-1}\mathbf{V}.$$



Review of ridge regression (polynomial form)

- Ridge regression in primal form (when $m > d' = \binom{p+d}{p}$)
  - **Learning/Training**:
    $$\mathbf{w}^* = (\mathbf{P}^\top \mathbf{P} + \lambda \mathbf{I})^{-1} \mathbf{P}^\top \mathbf{y}$$
  - **Prediction/Testing**: Given a new sample $\mathbf{x}_{\text{new}}$
    $$\hat{y}_{\text{new}} = \mathbf{p}_{\text{new}}^\top \mathbf{w}^*$$
    where $\mathbf{p}_{\text{new}}$ is the polynomial vector associated to $\mathbf{x}_{\text{new}}$.
- Ridge regression in dual form (when $m < d' = \binom{p+d}{p}$)
  - **Learning/Training**:
    $$\mathbf{w}^* = \mathbf{P}^\top (\mathbf{P}\mathbf{P}^\top + \lambda \mathbf{I})^{-1} \mathbf{y}$$
  - **Prediction/Testing**: Given a new sample $\mathbf{x}_{\text{new}}$
    $$\hat{y}_{\text{new}} = \mathbf{p}_{\text{new}}^\top \mathbf{w}^*$$



Regression/classification

- For regression applications:
  - Learn continuous-valued y by using either primal or dual forms
  - Prediction:

$$\hat{y}_{\text{new}} = \mathbf{p}_{\text{new}}^{\top} \mathbf{w}^{*}.$$

- For classification applications:
  - Learn discrete-valued $y \in \{-1, +1\}$ (for binary classification) or one-hot encoded $\mathbf{Y}$ (for $y \in \{1, 2, \ldots, C\}$ for multi-class classification) using either primal or dual forms
  - Binary prediction

$$\hat{y}_{\text{new}} = \text{sign}\left(\mathbf{p}_{\text{new}}^{\top} \mathbf{w}^{*}\right)$$

  - Multi-class prediction

$$\hat{y}_{\text{new}} = \underset{k \in \{1,2,\ldots,C\}}{\arg\max} \left(\mathbf{p}_{\text{new}}^{\top} \mathbf{W}^{*}[:, k]\right)$$



Useful Python packages and functions

- `sklearn.preprocessing PolynomialFeatures`,
- `np.sign`,
- `sklearn.model_selection train_test_split`,
- `sklearn.preprocessing OneHotEncoder`



[Exercise] $\mathbf{X}^\top\mathbf{X} + \lambda\mathbf{I}$ $(\forall\lambda \leq 0)$ is always invertible?

- We wish to solve

$$\overline{\mathbf{w}}^* = \underset{\overline{\mathbf{w}}=[b,\mathbf{w}]^\top}{\arg\min} \ (\mathbf{X}\overline{\mathbf{w}} - \mathbf{y})^\top(\mathbf{X}\overline{\mathbf{w}} - \mathbf{y}) + \lambda\overline{\mathbf{w}}^\top\overline{\mathbf{w}}.$$

- Expanding the objective, we obtain

$$(\mathbf{X}\overline{\mathbf{w}} - \mathbf{y})^\top(\mathbf{X}\overline{\mathbf{w}} - \mathbf{y}) + \lambda\overline{\mathbf{w}}^\top\overline{\mathbf{w}} = \overline{\mathbf{w}}^\top(\mathbf{X}^\top\mathbf{X} + \lambda\mathbf{I})\overline{\mathbf{w}} - 2\overline{\mathbf{w}}^\top(\mathbf{X}^\top\mathbf{y}) + \mathbf{y}^\top\mathbf{y}$$

- Differentiating w.r.t. $\overline{\mathbf{w}}$ and setting the result to zero yields

$$2(\mathbf{X}^\top\mathbf{X} + \lambda\mathbf{I})\overline{\mathbf{w}}^* = 2(\mathbf{X}^\top\mathbf{y}) \quad \Longleftrightarrow \quad \overline{\mathbf{w}}^* = (\mathbf{X}^\top\mathbf{X} + \lambda\mathbf{I})^{-1}\mathbf{X}^\top\mathbf{y}.$$

- For any $\lambda \leq 0$, $\mathbf{X}^\top\mathbf{X} + \lambda\mathbf{I}$ is always invertible?
- For any $\lambda < 0$, $\mathbf{X}^\top\mathbf{X} + \lambda\mathbf{I}$ is always invertible?
- Counter example?

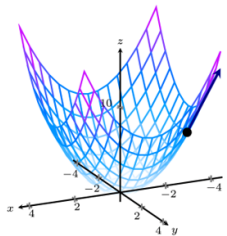


[Exercise] $\mathbf{X}^\top\mathbf{X} + \lambda\mathbf{I}$ $(\forall\lambda \leq 0)$ is always invertible?



Gradient descent (GD) algorithm: motivation

- Suppose we want to minimize $C(\mathbf{w})$ with respect to $\mathbf{w} = [w_1, \ldots, w_d]^\top$
- Gradient $\nabla_{\mathbf{w}} C(\mathbf{w}) = \begin{pmatrix} \frac{\partial C}{\partial w_1} \\ \vdots \\ \frac{\partial C}{\partial w_d} \end{pmatrix}$



  - Gradient $\nabla_{\mathbf{w}} C(\mathbf{w})$: vector & function of $\mathbf{w}$ (hard to find the solution of $\nabla_{\mathbf{w}} C(\mathbf{w})$)
  - $\nabla_{\mathbf{w}} C(\mathbf{w})$: direction at $\mathbf{w}$ where C is increasing most rapidly
    $-\nabla_{\mathbf{w}} C(\mathbf{w})$ is direction at $\mathbf{w}$ where C is decreasing most rapidly

⊙ Hint: Think about the case with $\mathbf{w} \in \mathbb{R}^1$.

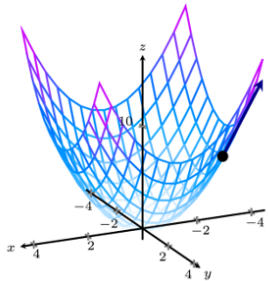
⊙ Book: 'Multivariable Calculus' by James Stewart



Gradient descent (GD) algorithm: basics

Algorithm 1: Gradient Descent (GD)

1: Initialize $\mathbf{w}_0$ and learning rate/step size $\eta > 0$
2: **while** true **do**
3: Compute $\mathbf{w}_{k+1} \leftarrow \mathbf{w}_k - \eta \nabla_{\mathbf{w}} C(\mathbf{w}_k)$
4: **if** converge **then**
5: **return** $\mathbf{w}_{k+1}$
6: **end if**
7: **end while**



◦ According to multi-variable calculus, if η is not too big, then $C(\mathbf{w}_{k+1}) < C(\mathbf{W}_k)$.

⇒ We get better $\mathbf{w}$ after each iteration.



Gradient descent (GD) algorithm: basics

- Possible convergence criteria
  - Set maximum iteration $k_{\max}$
  - Check percentage or absolute change in objective function C below a threshold
  - Check percentage or absolute change in parameter $\mathbf{w}$ below a threshold
- Limitation: Gradient descent can only find local minimum
  - Because gradient $= \mathbf{0}$ at local minimum

    so $\mathbf{w}$ won't change after that
- Impact of η
  - η is too small:
    - Converge slowly
  - η is too large:
    - Overshoot the local
    - Converge slowly to the minimum or may never converge



GD: choice of convergence criteria

- Set maximum iteration $k_{\max}$
- Check percentage or absolute change in objective function C below a threshold
  - Pro: direct measure of optimization progress, prevents unnecessary iterations
  - Con: computational cost, sensitive to scale, NOT guarantee convergence of $\mathbf{w}$
- Check percentage or absolute change in parameter $\mathbf{w}$ below a threshold
  - Pro: stable solution, less sensitive to oscillations in the function value
  - Con: can take longer to detect convergence, NOT directly reflect progress in objective function

⊙ Change of function C or parameter $\mathbf{w}$?

- Function C: care primarily about minimizing C, C is well-behaved (e.g., smooth and convex), low computational cost
- Parameter $\mathbf{w}$: care primarily if the parameters are stable and the optimization process has converged

○ Do hybrid?



Variation/convergence of gradient descent (GD)

- Standard GD: $\mathbf{w}_{k+1} \leftarrow \mathbf{w}_k - \eta \nabla_{\mathbf{w}} C(\mathbf{w}_k)$
- Changing learning rate:

  Decreasing learning rate, Adagrad (Adaptive Gradient Algorithm) with adaptive learning rate
- Different 'gradient':

  Momentum-based GD, Nesterov Accelerated Gradient (NAG)
- Design of cost function:

  Batched GD, Stochastic GD, Mini-Batch GD
- Adam (Adaptive Moment Estimation), Newton
- ...

⊙ Book: 'Convex Optimization' by Stephen Boyd and Lieven Vandenberghe

Available at https://stanford.edu/~boyd/cvxbook/



Variation of gradient descent (GD): changing learning rate

- Standard GD: $\mathbf{w}_{k+1} \leftarrow \mathbf{w}_k - \eta \nabla_{\mathbf{w}} C(\mathbf{w}_k)$
- Changing learning rate:

2. Adagrad (Adaptive Gradient Algorithm) with adaptive learning rate:

$$\mathbf{w}_{k+1} \leftarrow \mathbf{w}_k - \eta (M_k + \varepsilon I)^{-1} \nabla_{\mathbf{w}} C(\mathbf{w}_k)$$

- $M_k = \text{diag}(\sum_{s=0}^{k} \nabla_{\mathbf{w}} C(\mathbf{w}_s) \odot \nabla_{\mathbf{w}} C(\mathbf{w}_s))$ - a diagonal matrix where each diagonal entry is the sum of the element-wise squared gradients (i.e., $\nabla_{\mathbf{w}} C(\mathbf{w}_k) \odot \nabla_{\mathbf{w}} C(\mathbf{w}_k)$)
  $[a_1, a_2, a_3]^\top \odot [b_1, b_2, b_3]^\top = [a_1 b_1, a_2 b_2, a_3 b_3]^\top$
  ε: small constant to prevent division by zero
- Element-wise view

$$[\mathbf{w}_{k+1}]_i \leftarrow [\mathbf{w}_k]_i - \frac{\eta}{M_{ii,k} + \varepsilon} [\nabla_{\mathbf{w}} C(\mathbf{w}_k)]_i \ , \quad \text{where } M_{ii,k} = \sum_{s=0}^{k} [\nabla_{\mathbf{w}} C(\mathbf{w}_s)]_i^2$$



Variation of gradient descent (GD): changing learning rate

- Standard GD: $\mathbf{w}_{k+1} \leftarrow \mathbf{w}_k - \eta \nabla_{\mathbf{w}} C(\mathbf{w}_k)$
- Changing learning rate:

2. Adagrad (Adaptive Gradient Algorithm) with adaptive learning rate:

$$\mathbf{w}_{k+1} \leftarrow \mathbf{w}_k - \eta (M_k + \varepsilon I)^{-1} \nabla_{\mathbf{w}} C(\mathbf{w}_k)$$

- Pros: adjust the learning rate for based on the historical gradient information

  $\Rightarrow$ gives larger update to $\mathbf{w}_k$ with smaller gradient and smaller update to $\mathbf{w}_k$ with larger gradient.
- Cons: learning rate tends to shrink too quickly, leading to slow convergence or getting stuck at suboptimal points (vanishing learning rate).



Variation of gradient descent (GD): different 'gradient'

- Standard GD: $\mathbf{w}_{k+1} \leftarrow \mathbf{w}_k - \eta \nabla_{\mathbf{w}} C(\mathbf{w}_k)$
- Different 'gradient':

1. Momentum-based GD:

$$v_k \leftarrow \beta v_{k-1} + (1-\beta)\nabla_{\mathbf{w}} C(\mathbf{w}_k), \ \mathbf{w}_{k+1} \leftarrow \mathbf{w}_k - \eta v_k$$

β - momentum factor, $v_0 = \nabla_{\mathbf{w}} C(\mathbf{w}_0)$.

Why: maintaining a moving average of past gradients; converging fast but can overshoot the minimum



Variation of gradient descent (GD): different 'gradient'

- Standard GD: $\mathbf{w}_{k+1} \leftarrow \mathbf{w}_k - \eta \nabla_{\mathbf{w}} C(\mathbf{w}_k)$
- Different 'gradient':

2. Nesterov Accelerated Gradient (NAG):

$$v_t \leftarrow \beta v_{t-1} + \eta \nabla_{\mathbf{w}} C(\mathbf{w}_k - \beta v_{t-1}),\ \mathbf{w}_{k+1} \leftarrow \mathbf{w}_k - v_k$$

'Look ahead'! - anticipate the direction the optimizer will go in the next step



Variation of gradient descent (GD): different 'gradient'

- ⊙ Standard GD: $\mathbf{w}_{k+1} \leftarrow \mathbf{w}_k - \eta \nabla_{\mathbf{w}} C(\mathbf{w}_k)$
- ○ Different 'gradient':

2. Nesterov Accelerated Gradient (NAG):

$$v_t \leftarrow \beta v_{t-1} + \eta \nabla_{\mathbf{w}} C(\mathbf{w}_k - \beta v_{t-1}),\ \mathbf{w}_{k+1} \leftarrow \mathbf{w}_k - v_k$$

'Look ahead'! - anticipate the direction the optimizer will go in the next step

- Pros: faster convergence but slightly more complex
- Cons: slightly more complex than momentum alone



Variation of gradient descent (GD): view of dataset

- Standard GD: $\mathbf{w}_{k+1} \leftarrow \mathbf{w}_k - \eta \nabla_{\mathbf{w}} C(\mathbf{w}_k)$
- View of dataset:

1. Batch Gradient Descent (BGD):

   Loss function built using the **entire dataset** at each iteration.

   - Pros: converges smoothly to the minimum
   - Cons: computationally expensive, memory intensive, not ideal for online learning (hard to update the model incrementally as new data comes)

Algorithm 2: Batch Gradient Descent (BGD)

Initialize $\mathbf{w}_0$ and learning rate/step size $\eta > 0$
while true **do**
  Compute $\mathbf{w}_{k+1} \leftarrow \mathbf{w}_k - \eta \nabla_{\mathbf{w}} C(\mathbf{w}_k)$
  if converge **then**
    return $\mathbf{w}_{k+1}$
  end if
end while



Variation of gradient descent (GD): view of dataset

- Standard GD: $\mathbf{w}_{k+1} \leftarrow \mathbf{w}_k - \eta \nabla_{\mathbf{w}} C(\mathbf{w}_k)$
  - View of dataset:
2. Stochastic Gradient Descent (SGD):

   Loss function built using **one randomly chosen sample** at each iteration.

   - Pros: fast updates, suitable for online learning
   - Cons: noisy update, longer convergence time

Algorithm 3: Stochastic GD (SGD)

1: Initialize $\mathbf{w}_0$ and learning rate/step size $\eta > 0$
2: **while** true **do**
3: Randomly choose one sample m
4: Build the loss function $C_m(\mathbf{w})$ with the chosen sample m
5: Compute $\mathbf{w}_{k+1} \leftarrow \mathbf{w}_k - \eta \nabla_{\mathbf{w}} C_m(\mathbf{w}_k)$
6: **if** converge **then**
7: **return** $\mathbf{w}_{k+1}$
8: **end if**
9: **end while**



Variation of gradient descent (GD): view of dataset

- ⊙ Standard GD: $\mathbf{w}_{k+1} \leftarrow \mathbf{w}_k - \eta \nabla_{\mathbf{w}} C(\mathbf{w}_k)$
- ○ View of dataset:

3. Mini-Batch Gradient Descent (MBGD):

   Loss function built using **one randomly chosen sample** at each iteration.

- Pros: between SGD and BGD
- Cons: still has noise, hyperparameter tuning (choice of batch size)

Algorithm 4: Mini-Batch GD (MBGD)

1: Initialize $\mathbf{w}_0$ and learning rate/step size $\eta > 0$
2: Divide the entire dataset into small mini-batches
3: **while** true **do**
4: Randomly choose one small mini-batch n
5: Build the loss function $C_n(\mathbf{w})$ with the chosen mini-batch n
6: Compute $\mathbf{w}_{k+1} \leftarrow \mathbf{w}_k - \eta \nabla_{\mathbf{w}} C_n(\mathbf{w}_k)$
7: **if** converge **then**
8: **return** $\mathbf{w}_{k+1}$
9: **end if**
10: **end while**



Outline

1 Logistic regression

2 Support vector machines (SVM)

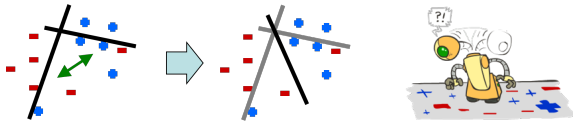


Possible issues for classification



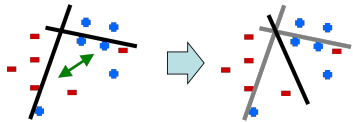
Possible issues for classification

- Noise: data is not separable



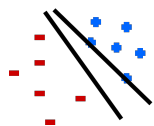
Possible issues for classification

- Noise: data is not separable





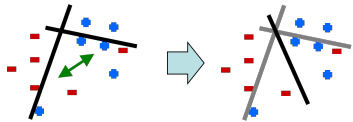
- Mediocre generalization:
  finds a "barely" separating solution





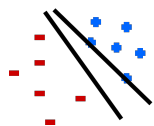
Possible issues for classification

- Noise: data is not separable



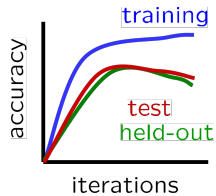


- Mediocre generalization:
  finds a "barely" separating solution



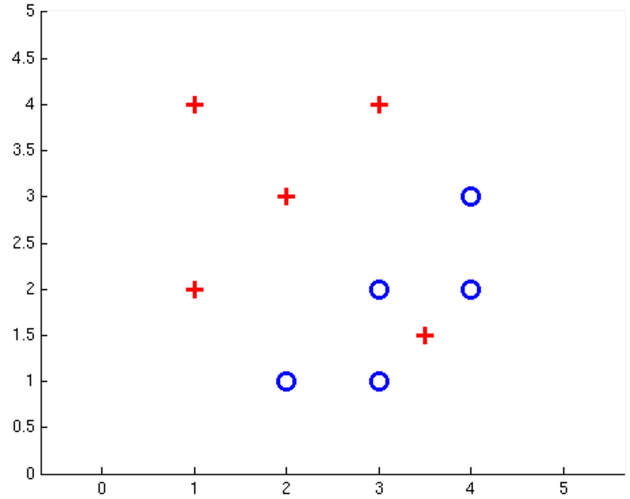


- Overtraining: test / held-out accuracy usually rises, then falls
  - Overtraining is a kind of overfitting





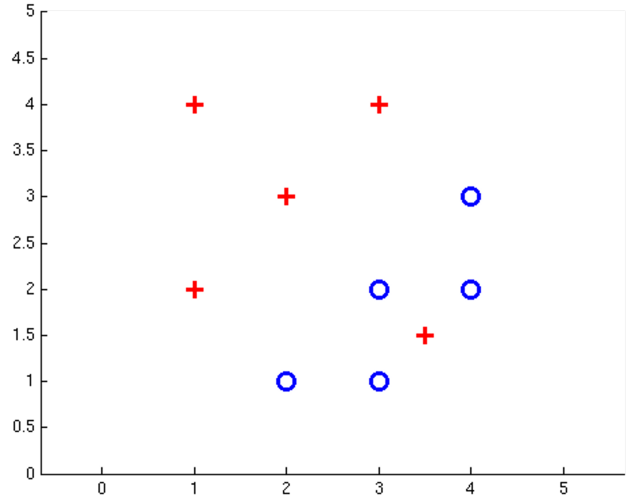
Non-separable case: deterministic decision





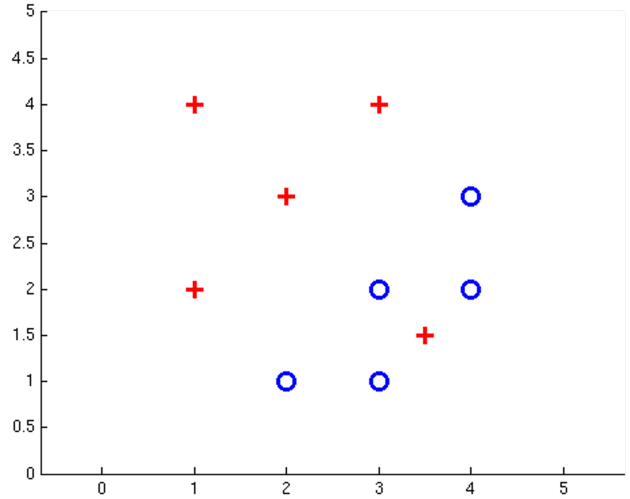
Non-separable case: deterministic decision

Even the best linear boundary makes at least one mistake





Non-separable case: probabilistic decision





Case: predicting the winner

- Deterministic decision: "I predict Team A will beat Team B."
- Probabilistic decision: "I predict Team A has a 55% chance of beating Team B."



Case: predicting the winner

- Deterministic decision: "I predict Team A will beat Team B."
- Probabilistic decision: "I predict Team A has a 55% chance of beating Team B."

- Sometimes we want to model our **confidence** in the labels, which may be noisy.
- Can be done through a **probability distribution** over the labels.



Motivation for logistic regression

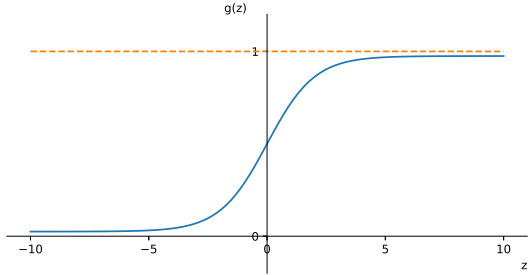
How to get probabilistic decisions?



Logistic regression

Logistic function/sigmoid function:

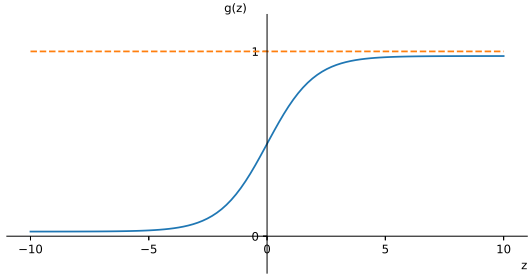
$$g(z) = \frac{1}{1 + \exp(-z)} \quad z \in \mathbb{R}. \tag{1.1}$$



Logistic regression

Logistic function/sigmoid function:

$$g(z) = \frac{1}{1+\exp(-z)} \quad z \in \mathbb{R}. \tag{1.1}$$



Logistic regression: given a sample $\mathbf{x}$ and the parameters $(\boldsymbol{\theta}, \theta_0)$, we set the probability of label $y = 1$ as

$$\Pr(y = 1 \mid \mathbf{x}, \boldsymbol{\theta}, \theta_0) = g(\langle \boldsymbol{\theta}, \mathbf{x} \rangle + \theta_0) = \frac{1}{1+\exp(-(\langle \boldsymbol{\theta}, \mathbf{x} \rangle + \theta_0))}. \tag{1.2}$$

This is known as the class conditional probability.



Logistic regression: Why?

One way to derive the form of the logistic function:

⊙ The **log-odds** of the predicted class probabilities should be a linear function of the inputs

$$\log \frac{\Pr(y = 1 \mid \mathbf{x}, \boldsymbol{\theta}, \theta_0)}{\Pr(y = -1 \mid \mathbf{x}, \boldsymbol{\theta}, \theta_0)} = \langle \boldsymbol{\theta}, \mathbf{x} \rangle + \theta_0.$$



Logistic regression: Why?

One way to derive the form of the logistic function:

⊙ The **log-odds** of the predicted class probabilities should be a linear function of the inputs

$$\log \frac{\Pr(y = 1 \mid \mathbf{x}, \boldsymbol{\theta}, \theta_0)}{\Pr(y = -1 \mid \mathbf{x}, \boldsymbol{\theta}, \theta_0)} = \langle \boldsymbol{\theta}, \mathbf{x} \rangle + \theta_0.$$

- Since $\Pr(y = 1 \mid E) = 1 - \Pr(y = -1 \mid E)$ (probabilities add to 1), we have

$$\log \frac{\Pr(y = 1 \mid \mathbf{x}, \boldsymbol{\theta}, \theta_0)}{1 - \Pr(y = 1 \mid \mathbf{x}, \boldsymbol{\theta}, \theta_0)} = \langle \boldsymbol{\theta}, \mathbf{x} \rangle + \theta_0.$$

- Solving this for $\Pr(y = 1 \mid \mathbf{x}, \boldsymbol{\theta}, \theta_0)$ yields the logistic regression model in (1.2) with logistic function in (1.1).



Logistic regression: Why?

$$\log \frac{\Pr(y=1 \mid \mathbf{x}, \boldsymbol{\theta}, \theta_0)}{\Pr(y=-1 \mid \mathbf{x}, \boldsymbol{\theta}, \theta_0)} = \langle \boldsymbol{\theta}, \mathbf{x} \rangle + \theta_0 \;\Rightarrow\; \Pr(y=1 \mid \mathbf{x}, \boldsymbol{\theta}, \theta_0) = \frac{1}{1+\exp\left(-\left(\langle \boldsymbol{\theta}, \mathbf{x} \rangle + \theta_0\right)\right)}.$$



Samples on the boundary

- Sample $\mathbf{x}$ sits on the decision boundary:

$$\langle \boldsymbol{\theta}, \mathbf{x} \rangle + \theta_0 = 0.$$

$\Rightarrow$ We are maximally uncertain about its label:

$$\Pr(y = 1 \mid \mathbf{x}, \boldsymbol{\theta}, \theta_0) = \Pr(y = -1 \mid \mathbf{x}, \boldsymbol{\theta}, \theta_0) = 1/2.$$



Samples on the boundary

- Sample $\mathbf{x}$ sits on the decision boundary:

$$\langle \boldsymbol{\theta}, \mathbf{x} \rangle + \theta_0 = 0.$$

$\Rightarrow$ We are maximally uncertain about its label:

$$\Pr(y = 1 \mid \mathbf{x}, \boldsymbol{\theta}, \theta_0) = \Pr(y = -1 \mid \mathbf{x}, \boldsymbol{\theta}, \theta_0) = 1/2.$$

- Logistic regression is known as a generalized linear model because the class conditional probabilities are a function (namely g) of a linear function of the sample $\mathbf{x}$

$$\Pr(y = 1 \mid \mathbf{x}, \boldsymbol{\theta}, \theta_0) = \frac{1}{1 + \exp\left(-\left(\langle \boldsymbol{\theta}, \mathbf{x} \rangle + \theta_0\right)\right)}.$$



Logistic regression: How?



Logistic regression: How?

⊙ Estimate parameters $(\boldsymbol{\theta}, \theta_0)$ with MLE.

- Given the logistic regression probabilistic model

$$\Pr\left(y_t \mid \mathbf{x}_t, \boldsymbol{\theta}, \theta_0\right) = g\left(y_t\left(\langle \boldsymbol{\theta}, \mathbf{x}_t \rangle + \theta_0\right)\right) = \frac{1}{1 + \exp\left(-y_t\left(\langle \boldsymbol{\theta}, \mathbf{x}_t \rangle + \theta_0\right)\right)}, \tag{1.3}$$

  we want to learn the parameters given the dataset $\mathcal{D} = \{(\mathbf{x}_t, y_t)_t\}$.



Logistic regression: How?

⊙ Estimate parameters $(\boldsymbol{\theta}, \theta_0)$ with MLE.

- Given the logistic regression probabilistic model

$$\Pr\left(y_t \mid \mathbf{x}_t, \boldsymbol{\theta}, \theta_0\right) = g\left(y_t\left(\langle \boldsymbol{\theta}, \mathbf{x}_t \rangle + \theta_0\right)\right) = \frac{1}{1 + \exp\left(-y_t\left(\langle \boldsymbol{\theta}, \mathbf{x}_t \rangle + \theta_0\right)\right)}, \tag{1.3}$$

  we want to learn the parameters given the dataset $\mathcal{D} = \{(\mathbf{x}_t, y_t)_t\}$.

- Maximize the probability of seeing the dataset $\mathcal{D}$ over all the parameters $(\boldsymbol{\theta}, \theta_0)$.
- Likelihood of $(\mathbf{x}_t, y_t)$ given $(\boldsymbol{\theta}, \theta_0)$: probability of seeing one sample $(\mathbf{x}_t, y_t)$ given that the current parameters are $(\boldsymbol{\theta}, \theta_0)$ is

$$L(\boldsymbol{\theta}, \theta_0 \mid \mathbf{x}_t, y_t) := \Pr(y_t \mid \mathbf{x}_t, \boldsymbol{\theta}, \theta_0) = g\big(y_t(\langle \mathbf{x}_t, \boldsymbol{\theta} \rangle + \theta_0)\big). \tag{1.4}$$



Likelihood of dataset

Likelihood of the dataset $\mathcal{D}$: overall likelihood of seeing the dataset given the parameters

$$L(\boldsymbol{\theta}, \theta_0 \mid \mathcal{D}) = \prod_{t=1}^{n} L(\boldsymbol{\theta}, \theta_0 \mid \mathbf{x}_t, y_t) = \prod_{t=1}^{n} \Pr(y_t \mid \mathbf{x}_t, \boldsymbol{\theta}, \theta_0). \tag{1.5}$$



Likelihood of dataset

Likelihood of the dataset $\mathcal{D}$: overall likelihood of seeing the dataset given the parameters

$$L(\boldsymbol{\theta}, \theta_0 \mid \mathcal{D}) = \prod_{t=1}^{n} L(\boldsymbol{\theta}, \theta_0 \mid \mathbf{x}_t, y_t) = \prod_{t=1}^{n} \Pr(y_t \mid \mathbf{x}_t, \boldsymbol{\theta}, \theta_0). \tag{1.5}$$

Is this a good formulation?

- Assumption: samples are independent.
- Usually there is little reason that data points are correlated.



Why MLE

- Obtain **maximum likelihood estimates (MLE)** of the parameters by maximizing this conditional likelihood with respect to $\boldsymbol{\theta}$ and θ_0.



Why MLE

- Obtain **maximum likelihood estimates (MLE)** of the parameters by maximizing this conditional likelihood with respect to $\boldsymbol{\theta}$ and θ_0.

- Assumptions:
  - Right model class is selected: e.g., logistic regression model.
  - Certain regularity conditions hold: e.g., $l2$-norm.

    'Likelihood': $L(\boldsymbol{\theta}, \theta_0 \mid \mathcal{D}) + \lambda ||\boldsymbol{\theta}||_2$



Why MLE

- Obtain **maximum likelihood estimates (MLE)** of the parameters by maximizing this conditional likelihood with respect to $\boldsymbol{\theta}$ and θ_0.
- Assumptions:
  - Right model class is selected: e.g., logistic regression model.
  - Certain regularity conditions hold: e.g., $l2$-norm.

    'Likelihood': $L(\boldsymbol{\theta}, \theta_0 \mid \mathcal{D}) + \lambda||\boldsymbol{\theta}||_2$
- MLE has many **nice** properties (considering $|\hat{\boldsymbol{\theta}}_n - \boldsymbol{\theta}|$):

$$\hat{\boldsymbol{\theta}}_n \to \boldsymbol{\theta} \text{ as } n \to \infty.$$

  - Consistent/unbiased with $l2$-norm (we will get the right parameter values in the limit of a large number of training examples);
  - Asymptotically normal;
  - Efficient (no other estimator will converge to the correct parameter values faster in the mean squared sense).



Find MLE of parameters: take logarithm

- Take the logarithm of the MLE problem: formulate the following optimization problem:

$$\max_{(\boldsymbol{\theta},\theta_0)\in\mathbb{R}^d\times\mathbb{R}} \sum_{t=1}^{n} \log \Pr(y_t \mid \mathbf{x}_t, \boldsymbol{\theta}, \theta_0) = \sum_{t=1}^{n} \log g\big(y_t(\langle \mathbf{x}_t, \boldsymbol{\theta}\rangle + \theta_0)\big). \tag{1.6}$$



Find MLE of parameters: take logarithm

- Take the logarithm of the MLE problem: formulate the following optimization problem:

$$\max_{(\boldsymbol{\theta},\theta_0)\in\mathbb{R}^d\times\mathbb{R}} \sum_{t=1}^{n} \log \Pr(y_t \mid \mathbf{x}_t, \boldsymbol{\theta}, \theta_0) = \sum_{t=1}^{n} \log g\big(y_t(\langle \mathbf{x}_t, \boldsymbol{\theta}\rangle + \theta_0)\big). \tag{1.6}$$

- Convert to minimization:

$$\arg\max_{(\boldsymbol{\theta},\theta_0)\in\mathbb{R}^d\times\mathbb{R}} L(\boldsymbol{\theta}, \theta_0 \mid \mathcal{D}) \equiv \arg\min_{(\boldsymbol{\theta},\theta_0)\in\mathbb{R}^d\times\mathbb{R}} \sum_{t=1}^{n} -\log \Pr(y_t \mid \mathbf{x}_t, \boldsymbol{\theta}, \theta_0) \tag{1.7}$$



Find MLE of parameters: take logarithm

- Take the logarithm of the MLE problem: formulate the following optimization problem:

$$\max_{(\boldsymbol{\theta},\theta_0)\in\mathbb{R}^d\times\mathbb{R}} \sum_{t=1}^{n} \log \Pr(y_t \mid \mathbf{x}_t, \boldsymbol{\theta}, \theta_0) = \sum_{t=1}^{n} \log g\big(y_t(\langle \mathbf{x}_t, \boldsymbol{\theta}\rangle + \theta_0)\big). \tag{1.6}$$

- Convert to minimization:

$$\arg\max_{(\boldsymbol{\theta},\theta_0)\in\mathbb{R}^d\times\mathbb{R}} L(\boldsymbol{\theta}, \theta_0 \mid \mathcal{D}) \equiv \arg\min_{(\boldsymbol{\theta},\theta_0)\in\mathbb{R}^d\times\mathbb{R}} \sum_{t=1}^{n} -\log \Pr(y_t \mid \mathbf{x}_t, \boldsymbol{\theta}, \theta_0) \tag{1.7}$$

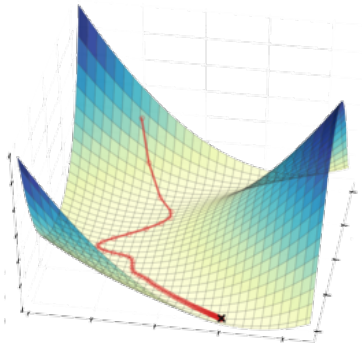
- Equivalent to minimizing:

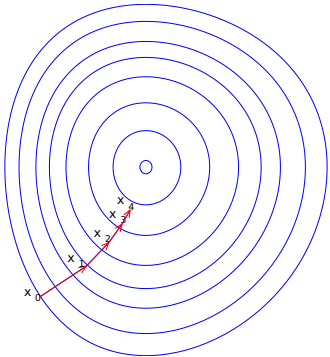
$$\sum_{t=1}^{n} -\log g\big(y_t(\langle \mathbf{x}_t, \boldsymbol{\theta}\rangle + \theta_0)\big) = \sum_{t=1}^{n} \log\big[1 + \exp(-y_t(\langle \mathbf{x}_t, \boldsymbol{\theta}\rangle + \theta_0))\big]. \tag{1.8}$$



Find MLE of parameters: stochastic gradient descent (SGD)

- Optimize (1.8) using calculus with a **stochastic gradient descent (SGD)** algorithm.







Find MLE of parameters: stochastic gradient descent (SGD)

- Take the gradients with respect to $\boldsymbol{\theta}$ and θ_0 separately

$$\begin{aligned}\frac{\mathrm{d}}{\mathrm{d}\theta_0}\log\left[1+\exp(-y_t(\langle \mathbf{x}_t,\boldsymbol{\theta}\rangle+\theta_0))\right] &= -y_t\frac{\exp\left(-y_t(\langle \mathbf{x}_t,\boldsymbol{\theta}\rangle+\theta_0)\right)}{1+\exp\left(-y_t(\langle \mathbf{x}_t,\boldsymbol{\theta}\rangle+\theta_0)\right)}\\ &= -y_t\left[1-\Pr(y_t \mid \mathbf{x}_t,\boldsymbol{\theta},\theta_0)\right], \qquad (1.9)\\ \frac{\mathrm{d}}{\mathrm{d}\boldsymbol{\theta}}\log\left[1+\exp(-y_t(\langle \mathbf{x}_t,\boldsymbol{\theta}\rangle+\theta_0))\right] &= -y_t\mathbf{x}_t\left[1-\Pr(y_t \mid \mathbf{x}_t,\boldsymbol{\theta},\theta_0)\right]. \qquad (1.10)\end{aligned}$$



Find MLE of parameters: stochastic gradient descent (SGD)

- Take the gradients with respect to $\boldsymbol{\theta}$ and θ_0 separately

$$\frac{\mathrm{d}}{\mathrm{d}\theta_0} \log\big[1 + \exp(-y_t(\langle \mathbf{x}_t, \boldsymbol{\theta}\rangle + \theta_0))\big] = -y_t \frac{\exp\big(-y_t(\langle \mathbf{x}_t, \boldsymbol{\theta}\rangle + \theta_0)\big)}{1 + \exp\big(-y_t(\langle \mathbf{x}_t, \boldsymbol{\theta}\rangle + \theta_0)\big)}$$
$$= -y_t\big[1 - \Pr(y_t \mid \mathbf{x}_t, \boldsymbol{\theta}, \theta_0)\big], \tag{1.9}$$

$$\frac{\mathrm{d}}{\mathrm{d}\boldsymbol{\theta}} \log\big[1 + \exp(-y_t(\langle \mathbf{x}_t, \boldsymbol{\theta}\rangle + \theta_0))\big] = -y_t \mathbf{x}_t\big[1 - \Pr(y_t \mid \mathbf{x}_t, \boldsymbol{\theta}, \theta_0)\big]. \tag{1.10}$$

⊙ **Parameter update:** by selecting training examples at random and moving the parameters in the opposite direction of the derivatives ($\eta > 0$: small learning rate)

$$\boldsymbol{\theta} \longleftarrow \boldsymbol{\theta} + \eta \cdot y_t \mathbf{x}_t\big[1 - \Pr(y_t \mid \mathbf{x}_t, \boldsymbol{\theta}, \theta_0)\big] \tag{1.11}$$
$$\theta_0 \longleftarrow \theta_0 + \eta \cdot y_t\big[1 - \Pr(y_t \mid \mathbf{x}_t, \boldsymbol{\theta}, \theta_0)\big]. \tag{1.12}$$



Find MLE of parameters: stochastic gradient descent (SGD)

- Setting the gradient to zero is a necessary condition for the optimality of $(\boldsymbol{\theta}^*, \theta_0^*)$

$$\frac{\mathrm{d}}{\mathrm{d}\theta_0}\ell(\boldsymbol{\theta}, \theta_0 \mid \mathcal{D}) = \sum_{t=1}^{n} -y_t\big[1 - \mathsf{Pr}(y_t \mid \mathbf{x}_t, \boldsymbol{\theta}^*, \theta_0^*)\big] = 0, \tag{1.13}$$

$$\frac{\mathrm{d}}{\mathrm{d}\boldsymbol{\theta}}\ell(\boldsymbol{\theta}, \theta_0 \mid \mathcal{D}) = \sum_{t=1}^{n} -y_t\mathbf{x}_t\big[1 - \mathsf{Pr}(y_t \mid \mathbf{x}_t, \boldsymbol{\theta}^*, \theta_0^*)\big] = \mathbf{0}. \tag{1.14}$$



Find MLE of parameters: stochastic gradient descent (SGD)

- Setting the gradient to zero is a necessary condition for the optimality of $(\boldsymbol{\theta}^*, \theta_0^*)$

$$\frac{\mathrm{d}}{\mathrm{d}\theta_0}\ell(\boldsymbol{\theta}, \theta_0 \mid \mathcal{D}) = \sum_{t=1}^{n} -y_t\big[1 - \Pr(y_t \mid \mathbf{x}_t, \boldsymbol{\theta}^*, \theta_0^*)\big] = 0, \tag{1.13}$$

$$\frac{\mathrm{d}}{\mathrm{d}\boldsymbol{\theta}}\ell(\boldsymbol{\theta}, \theta_0 \mid \mathcal{D}) = \sum_{t=1}^{n} -y_t\mathbf{x}_t\big[1 - \Pr(y_t \mid \mathbf{x}_t, \boldsymbol{\theta}^*, \theta_0^*)\big] = \mathbf{0}. \tag{1.14}$$

- SGD leads to no significant change on average when the gradient of the full objective equals zero.



Find MLE of parameters: stochastic gradient descent (SGD)

- Setting the gradient to zero is a necessary condition for the optimality of $(\boldsymbol{\theta}^*, \theta_0^*)$

$$\frac{\mathrm{d}}{\mathrm{d}\theta_0}\ell(\boldsymbol{\theta}, \theta_0 \mid \mathcal{D}) = \sum_{t=1}^{n} -y_t\big[1 - \Pr(y_t \mid \mathbf{x}_t, \boldsymbol{\theta}^*, \theta_0^*)\big] = 0, \tag{1.13}$$

$$\frac{\mathrm{d}}{\mathrm{d}\boldsymbol{\theta}}\ell(\boldsymbol{\theta}, \theta_0 \mid \mathcal{D}) = \sum_{t=1}^{n} -y_t\mathbf{x}_t\big[1 - \Pr(y_t \mid \mathbf{x}_t, \boldsymbol{\theta}^*, \theta_0^*)\big] = \mathbf{0}. \tag{1.14}$$

- SGD leads to no significant change on average when the gradient of the full objective equals zero.

⊙ Sum in (1.13): Difference between mistake probabilities associated with positively and negatively labeled examples.
⊙ Optimality of θ_0^*: Mistakes are balanced in this (soft) sense.



Understand $\boldsymbol{\theta}^*$ and θ_0^* (optional)

- Optimality of θ_0^*: vector of optimal prediction errors is orthogonal to the vector of labels.
- Optimality of $\boldsymbol{\theta}^*$: vector of optimal prediction errors is orthogonal to all linear functions of the inputs.



Understand $\boldsymbol{\theta}^*$ and θ_0^* (optional)

$\odot$ Optimality of θ_0^*: vector of optimal prediction errors is orthogonal to the vector of labels.
$\odot$ Optimality of $\boldsymbol{\theta}^*$: vector of optimal prediction errors is orthogonal to all linear functions of the inputs.

- Map ± 1 labels to $\{0, 1\}$ with $\tilde{y}_t = (1 + y_t)/2$:

$$\frac{\mathrm{d}}{\mathrm{d}\theta_0}\ell(\boldsymbol{\theta}, \theta_0 \mid \mathcal{D}) = \sum_{t=1}^{n} -y_t\big[1 - \mathsf{Pr}(y_t \mid \mathbf{x}_t, \boldsymbol{\theta}^*, \theta_0^*)\big] = \sum_{t=1}^{n} \big[\tilde{y}_t - \mathsf{Pr}(y = 1 \mid \mathbf{x}_t, \boldsymbol{\theta}^*, \theta_0^*)\big] = 0,$$

$$\frac{\mathrm{d}}{\mathrm{d}\boldsymbol{\theta}}\ell(\boldsymbol{\theta}, \theta_0 \mid \mathcal{D}) = \sum_{t=1}^{n} -y_t\mathbf{x}_t\big[1 - \mathsf{Pr}(y_t \mid \mathbf{x}_t, \boldsymbol{\theta}^*, \theta_0^*)\big] = \sum_{t=1}^{n} \mathbf{x}_t\big[\tilde{y}_t - \mathsf{Pr}(y = 1 \mid \mathbf{x}_t, \boldsymbol{\theta}^*, \theta_0^*)\big] = \mathbf{0}.$$



Understand $\boldsymbol{\theta}^*$ and θ_0^* (optional)

⊙ Optimality of θ_0^*: vector of optimal prediction errors is orthogonal to the vector of labels.
⊙ Optimality of $\boldsymbol{\theta}^*$: vector of optimal prediction errors is orthogonal to all linear functions of the inputs.

- Map ± 1 labels to $\{0, 1\}$ with $\tilde{y}_t = (1 + y_t)/2$:

$$\frac{\mathrm{d}}{\mathrm{d}\theta_0}\ell(\boldsymbol{\theta}, \theta_0 \mid \mathcal{D}) = \sum_{t=1}^{n} -y_t\big[1 - \Pr(y_t \mid \mathbf{x}_t, \boldsymbol{\theta}^*, \theta_0^*)\big] = \sum_{t=1}^{n} \big[\tilde{y}_t - \Pr(y = 1 \mid \mathbf{x}_t, \boldsymbol{\theta}^*, \theta_0^*)\big] = 0,$$

$$\frac{\mathrm{d}}{\mathrm{d}\boldsymbol{\theta}}\ell(\boldsymbol{\theta}, \theta_0 \mid \mathcal{D}) = \sum_{t=1}^{n} -y_t\mathbf{x}_t\big[1 - \Pr(y_t \mid \mathbf{x}_t, \boldsymbol{\theta}^*, \theta_0^*)\big] = \sum_{t=1}^{n} \mathbf{x}_t\big[\tilde{y}_t - \Pr(y = 1 \mid \mathbf{x}_t, \boldsymbol{\theta}^*, \theta_0^*)\big] = \mathbf{0}.$$

- Mistake probability: $1 - \Pr(y_t \mid \mathbf{x}_t, \boldsymbol{\theta}^*, \theta_0^*)$
- Prediction error: $\tilde{y}_t - \Pr(y = 1 \mid \mathbf{x}_t, \boldsymbol{\theta}^*, \theta_0^*)$



Understand $\boldsymbol{\theta}^*$ and θ_0^* (optional)

$\odot$ Two optimality conditions: optimal prediction errors are orthogonal to any linear function of the inputs, i.e., for any $(\tilde{\boldsymbol{\theta}}, \tilde{\theta}_0)$,

$$\tilde{\theta}_0 \sum_{t=1}^{n} \left[\tilde{y}_t - \Pr(y = 1 \mid \mathbf{x}_t, \boldsymbol{\theta}^*, \theta_0^*)\right] + \left\langle \tilde{\boldsymbol{\theta}}, \sum_{t=1}^{n} \mathbf{x}_t \left[\tilde{y}_t - \Pr(y = 1 \mid \mathbf{x}_t, \boldsymbol{\theta}^*, \theta_0^*)\right] \right\rangle = 0, \quad (1.15)$$

or

$$\sum_{t=1}^{n} \left(\tilde{\theta}_0 + \langle \tilde{\boldsymbol{\theta}}, \mathbf{x}_t \rangle\right) \left[\tilde{y}_t - \Pr(y = 1 \mid \mathbf{x}_t, \boldsymbol{\theta}^*, \theta_0^*)\right] = 0. \quad (1.16)$$



Understand $\boldsymbol{\theta}^*$ and θ_0^* (optional)

$\odot$ Vector of optimal prediction errors $\mathbf{e} := (\tilde{y}_t - \Pr(y = 1 \mid \mathbf{x}_t, \boldsymbol{\theta}^*, \theta_0^*))_{t=1}^n$ is orthogonal to any linear function of the inputs (data) $\mathbf{d} := (\langle \tilde{\boldsymbol{\theta}}, \mathbf{x}_t \rangle + \tilde{\theta}_0)_{t=1}^n$.

$\Rightarrow$ There is no further linearly available information in the training samples $\{\mathbf{x}_t\}_{t=1}^n$ to improve the prediction probabilities (or mistake probabilities).



Understand θ^* and θ_0^* (optional)

$\odot$ Vector of optimal prediction errors $\mathbf{e} := (\tilde{y}_t - \Pr(y = 1 \mid \mathbf{x}_t, \boldsymbol{\theta}^*, \theta_0^*))_{t=1}^n$ is orthogonal to any linear function of the inputs (data) $\mathbf{d} := (\langle \tilde{\boldsymbol{\theta}}, \mathbf{x}_t \rangle + \tilde{\theta}_0)_{t=1}^n$.

$\Rightarrow$ There is no further linearly available information in the training samples $\{\mathbf{x}_t\}_{t=1}^n$ to improve the prediction probabilities (or mistake probabilities).

Theorem 1.1

At the optimal solution for the parameter and offset vector $\boldsymbol{\theta}^*, \theta_0^*$, the prediction error vector $\mathbf{e} := (\tilde{y}_t - \Pr(y = 1|\mathbf{x}_t, \boldsymbol{\theta}^*, \theta_0^*))_{t=1}^n$ is orthogonal to any linear function of the data, i.e., for any $\bar{\boldsymbol{\theta}}, \bar{\theta}_0$,

$$\langle \mathbf{z}, \mathbf{e} \rangle = 0, \quad \textit{where} \quad \mathbf{z} := (\langle \bar{\boldsymbol{\theta}}, \mathbf{x}_t \rangle + \bar{\theta}_0)_{t=1}^n. \tag{1.17}$$

Furthermore if a prediction vector $\mathbf{e}$ satisfies (1.17), it has the **minimal ℓ_2 norm** *of $(\tilde{y}_t - \Pr(y = 1|\mathbf{x}_t, \boldsymbol{\theta}, \theta_0))_{t=1}^n$ over all $\boldsymbol{\theta}, \theta_0$.*



Understand θ^* and θ_0^* (optional)

$\odot$ Vector of optimal prediction errors $\mathbf{e} := (\tilde{y}_t - \Pr(y = 1 \mid \mathbf{x}_t, \boldsymbol{\theta}^*, \theta_0^*))_{t=1}^n$ is orthogonal to any linear function of the inputs (data) $\mathbf{d} := (\langle \tilde{\boldsymbol{\theta}}, \mathbf{x}_t \rangle + \tilde{\theta}_0)_{t=1}^n$.

$\Rightarrow$ There is no further linearly available information in the training samples $\{\mathbf{x}_t\}_{t=1}^n$ to improve the prediction probabilities (or mistake probabilities).

Theorem 1.1

At the optimal solution for the parameter and offset vector $\boldsymbol{\theta}^*, \theta_0^*$, the prediction error vector $\mathbf{e} := (\tilde{y}_t - \Pr(y = 1|\mathbf{x}_t, \boldsymbol{\theta}^*, \theta_0^*))_{t=1}^n$ is orthogonal to any linear function of the data, i.e., for any $\bar{\boldsymbol{\theta}}, \bar{\theta}_0$,

$$\langle \mathbf{z}, \mathbf{e} \rangle = 0, \quad \textit{where} \quad \mathbf{z} := (\langle \bar{\boldsymbol{\theta}}, \mathbf{x}_t \rangle + \bar{\theta}_0)_{t=1}^n. \tag{1.17}$$

Furthermore if a prediction vector $\mathbf{e}$ satisfies (1.17), it has the **minimal ℓ_2 norm** *of $(\tilde{y}_t - \Pr(y = 1|\mathbf{x}_t, \boldsymbol{\theta}, \theta_0))_{t=1}^n$ over all $\boldsymbol{\theta}, \theta_0$.*

Hint for proof: Assume $\langle \mathbf{z}, \mathbf{e} \rangle \neq 0$, we can prove $\mathbf{e}' := \mathbf{e} - \dfrac{\langle \mathbf{z}, \mathbf{e} \rangle}{\langle \mathbf{z}, \mathbf{z} \rangle} \mathbf{z}$ satisfies $\|\mathbf{e}'\| < \|\mathbf{e}\|$.



Procedure in logistic regression

- Given the logistic regression probabilistic model

$$\Pr\left(y_t \mid \mathbf{x}_t, \boldsymbol{\theta}, \theta_0\right) = g\left(y_t\left(\langle \boldsymbol{\theta}, \mathbf{x}_t \rangle + \theta_0\right)\right) = \frac{1}{1 + \exp\left(-y_t\left(\langle \boldsymbol{\theta}, \mathbf{x}_t \rangle + \theta_0\right)\right)}, \tag{1.3}$$

- Maximize the likelihood of the dataset $\mathcal{D}$

$$L(\boldsymbol{\theta}, \theta_0 \mid \mathcal{D}) = \prod_{t=1}^{n} L(\boldsymbol{\theta}, \theta_0 \mid \mathbf{x}_t, y_t) = \prod_{t=1}^{n} \Pr(y_t \mid \mathbf{x}_t, \boldsymbol{\theta}, \theta_0). \tag{1.18}$$

- Take the logarithm of the MLE problem

$$\max_{(\boldsymbol{\theta}, \theta_0) \in \mathbb{R}^d \times \mathbb{R}} \sum_{t=1}^{n} \log \Pr(y_t \mid \mathbf{x}_t, \boldsymbol{\theta}, \theta_0) \;\Leftrightarrow\; \min_{(\boldsymbol{\theta}, \theta_0) \in \mathbb{R}^d \times \mathbb{R}} \sum_{t=1}^{n} \log\left[1 + \exp(-y_t(\langle \mathbf{x}_t, \boldsymbol{\theta} \rangle + \theta_0))\right]. \tag{1.19}$$

Procedure in logistic regression

• SGD: Take the gradients with respect to $\boldsymbol{\theta}$ and θ_0 separately and update parameters $\boldsymbol{\theta}$ and θ_0 ($\eta > 0$: small learning rate)

$$\boldsymbol{\theta} \longleftarrow \boldsymbol{\theta} + \eta \cdot y_t \mathbf{x}_t \big[1 - \Pr(y_t \mid \mathbf{x}_t, \boldsymbol{\theta}, \theta_0)\big] \tag{1.20}$$

$$\theta_0 \longleftarrow \theta_0 + \eta \cdot y_t \big[1 - \Pr(y_t \mid \mathbf{x}_t, \boldsymbol{\theta}, \theta_0)\big]. \tag{1.21}$$



Regularization in logistic regression

- The likelihood is strictly increasing as a function of $y_t(\langle \boldsymbol{\theta}^*, \mathbf{x}_t \rangle + \theta_0^*)$.

$$L(\boldsymbol{\theta}, \theta_0 \mid \mathcal{D}) = \prod_{t=1}^{n} \Pr(y_t \mid \mathbf{x}_t, \boldsymbol{\theta}, \theta_0) = \prod_{t=1}^{n} \frac{1}{1 + \exp\left(-y_t \left(\langle \boldsymbol{\theta}, \mathbf{x}_t \rangle + \theta_0\right)\right)}$$



Regularization in logistic regression

⊙ The likelihood is strictly increasing as a function of $y_t(\langle \boldsymbol{\theta}^*, \mathbf{x}_t \rangle + \theta_0^*)$.

$$L(\boldsymbol{\theta}, \theta_0 \mid \mathcal{D}) = \prod_{t=1}^{n} \Pr(y_t \mid \mathbf{x}_t, \boldsymbol{\theta}, \theta_0) = \prod_{t=1}^{n} \frac{1}{1 + \exp\left(-y_t \left(\langle \boldsymbol{\theta}, \mathbf{x}_t \rangle + \theta_0\right)\right)}$$

- Infinite scaling of any parameters corresponding to a perfect linear classifier would attain the highest likelihood.
  - If the training data are linearly- or affinely-separable, we can find parameters $(\boldsymbol{\theta}^*, \theta_0^*)$ such that $y_t(\langle \boldsymbol{\theta}^*, \mathbf{x}_t \rangle + \theta_0^*) \geq 0$ for all t.



Regularization in logistic regression

⊙ The likelihood is strictly increasing as a function of $y_t(\langle \boldsymbol{\theta}^*, \mathbf{x}_t \rangle + \theta_0^*)$.

$$L(\boldsymbol{\theta}, \theta_0 \mid \mathcal{D}) = \prod_{t=1}^{n} \Pr(y_t \mid \mathbf{x}_t, \boldsymbol{\theta}, \theta_0) = \prod_{t=1}^{n} \frac{1}{1 + \exp\left(-y_t \left(\langle \boldsymbol{\theta}, \mathbf{x}_t \rangle + \theta_0\right)\right)}$$

- Infinite scaling of any parameters corresponding to a perfect linear classifier would attain the highest likelihood.
  - If the training data are linearly- or affinely-separable, we can find parameters $(\boldsymbol{\theta}^*, \theta_0^*)$ such that $y_t(\langle \boldsymbol{\theta}^*, \mathbf{x}_t \rangle + \theta_0^*) \geq 0$ for all t.

⊙ **Q:** Why?



Regularization in logistic regression

⊙ The likelihood is strictly increasing as a function of $y_t(\langle \boldsymbol{\theta}^*, \mathbf{x}_t \rangle + \theta_0^*)$.

$$L(\boldsymbol{\theta}, \theta_0 \mid \mathcal{D}) = \prod_{t=1}^{n} \Pr(y_t \mid \mathbf{x}_t, \boldsymbol{\theta}, \theta_0) = \prod_{t=1}^{n} \frac{1}{1 + \exp\left(-y_t \left(\langle \boldsymbol{\theta}, \mathbf{x}_t \rangle + \theta_0\right)\right)}$$

- Infinite scaling of any parameters corresponding to a perfect linear classifier would attain the highest likelihood.
  - If the training data are linearly- or affinely-separable, we can find parameters $(\boldsymbol{\theta}^*, \theta_0^*)$ such that $y_t(\langle \boldsymbol{\theta}^*, \mathbf{x}_t \rangle + \theta_0^*) \geq 0$ for all t.
    Hint: By scaling up the parameters, we make these values larger and larger.

⊙ **Q:** Why?



Regularization in logistic regression

⊙ The likelihood is strictly increasing as a function of $y_t(\langle \boldsymbol{\theta}^*, \mathbf{x}_t \rangle + \theta_0^*)$.

$$L(\boldsymbol{\theta}, \theta_0 \mid \mathcal{D}) = \prod_{t=1}^{n} \Pr(y_t \mid \mathbf{x}_t, \boldsymbol{\theta}, \theta_0) = \prod_{t=1}^{n} \frac{1}{1 + \exp\left(-y_t \left(\langle \boldsymbol{\theta}, \mathbf{x}_t \rangle + \theta_0\right)\right)}$$

- Infinite scaling of any parameters corresponding to a perfect linear classifier would attain the highest likelihood.
  - If the training data are linearly- or affinely-separable, we can find parameters $(\boldsymbol{\theta}^*, \theta_0^*)$ such that $y_t(\langle \boldsymbol{\theta}^*, \mathbf{x}_t \rangle + \theta_0^*) \geq 0$ for all t.
    Hint: By scaling up the parameters, we make these values larger and larger.

⊙ **Q:** Why?

A: $\log \dfrac{\Pr\left(y = 1 \mid \mathbf{x}, \boldsymbol{\theta}, \theta_0\right)}{\Pr\left(y = -1 \mid \mathbf{x}, \boldsymbol{\theta}, \theta_0\right)} = \langle \boldsymbol{\theta}, \mathbf{x} \rangle + \theta_0.$



Regularization in logistic regression

- The likelihood is strictly increasing as a function of $y_t(\langle \boldsymbol{\theta}^*, \mathbf{x}_t \rangle + \theta_0^*)$.

$$L(\boldsymbol{\theta}, \theta_0 \mid \mathcal{D}) = \prod_{t=1}^{n} \Pr(y_t \mid \mathbf{x}_t, \boldsymbol{\theta}, \theta_0) = \prod_{t=1}^{n} \frac{1}{1 + \exp\left(-y_t\left(\langle \boldsymbol{\theta}, \mathbf{x}_t \rangle + \theta_0\right)\right)}$$

- Infinite scaling of any parameters corresponding to a perfect linear classifier would attain the highest likelihood.
  - If the training data are linearly- or affinely-separable, we can find parameters $(\boldsymbol{\theta}^*, \theta_0^*)$ such that $y_t(\langle \boldsymbol{\theta}^*, \mathbf{x}_t \rangle + \theta_0^*) \geq 0$ for all t. By scaling up the parameters, we make these values larger and larger.
- Regularization: e.g., ℓ_2-norm

$$\min_{(\boldsymbol{\theta}, \theta_0) \in \mathbb{R}^d \times \mathbb{R}} \frac{\lambda}{2} \|\boldsymbol{\theta}\|^2 + \sum_{t=1}^{n} \log\left[1 + \exp(-y_t(\langle \mathbf{x}_t, \boldsymbol{\theta} \rangle + \theta_0))\right]. \tag{1.22}$$



Multi-class logistic regression

⊙ When we have $M \geq 2$ classes, replace

$$\Pr(y = 1 \mid \mathbf{x}, \boldsymbol{\theta}, \theta_0) = \frac{1}{1 + \exp\left(-\left(\langle \boldsymbol{\theta}, \mathbf{x} \rangle + \theta_0\right)\right)} \tag{1.2}$$

by the soft-max function

$$\Pr(y = c \mid \mathbf{x}, \{\boldsymbol{\theta}_c, \theta_{0,c}\}_{c=1}^{M}) = \frac{\exp(\langle \boldsymbol{\theta}_c, \mathbf{x} \rangle + \theta_{0,c})}{\sum_{c'=1}^{M} \exp(\langle \boldsymbol{\theta}_{c'}, \mathbf{x} \rangle + \theta_{0,c'})} \tag{1.23}$$

where we now have a different pair $(\boldsymbol{\theta}_c, \theta_{0,c})$ for each class.



Multi-class logistic regression

⊙ When we have $M \geq 2$ classes, replace

$$\Pr(y = 1 \mid \mathbf{x}, \boldsymbol{\theta}, \theta_0) = \frac{1}{1 + \exp\left(-\left(\langle \boldsymbol{\theta}, \mathbf{x} \rangle + \theta_0\right)\right)} \tag{1.2}$$

by the soft-max function

$$\Pr(y = c \mid \mathbf{x}, \{\boldsymbol{\theta}_c, \theta_{0,c}\}_{c=1}^{M}) = \frac{\exp(\langle \boldsymbol{\theta}_c, \mathbf{x} \rangle + \theta_{0,c})}{\sum_{c'=1}^{M} \exp(\langle \boldsymbol{\theta}_{c'}, \mathbf{x} \rangle + \theta_{0,c'})} \tag{1.23}$$

where we now have a different pair $(\boldsymbol{\theta}_c, \theta_{0,c})$ for each class.

• Without loss of generality, we can assume that one of the classes (say the first one) has $(\boldsymbol{\theta}_1, \theta_{0,1}) = (\mathbf{0}, 0)$, which is useful for showing that (1.2) is a special case of (1.27).
  ◦ **Why?**



[Exercise] Multi-class logistic regression

⊙ When we have $M \geq 2$ classes, replace

$$\Pr(y = 1 \mid \mathbf{x}, \boldsymbol{\theta}, \theta_0) = \frac{1}{1 + \exp\left(-\left(\langle \boldsymbol{\theta}, \mathbf{x} \rangle + \theta_0\right)\right)} \tag{1.2}$$

by the soft-max function

$$\Pr(y = c \mid \mathbf{x}, \{\boldsymbol{\theta}_c, \theta_{0,c}\}_{c=1}^{M}) = \frac{\exp(\langle \boldsymbol{\theta}_c, \mathbf{x} \rangle + \theta_{0,c})}{\sum_{c'=1}^{M} \exp(\langle \boldsymbol{\theta}_{c'}, \mathbf{x} \rangle + \theta_{0,c'})} \tag{1.27}$$

where we now have a different pair $(\boldsymbol{\theta}_c, \theta_{0,c})$ for each class.

- **Q:** show that (1.2) is a special case of (1.27).

○ Hint: without loss of generality, we can assume that one of the classes (say the first one) has $(\boldsymbol{\theta}_1, \theta_{0,1}) = (\mathbf{0}, 0)$.



Python demo: logistic regression

Canvas: DSAA2011-L4_supervised_c_demo_logistic_reg.pdf



Summary of logistic regression

- Given the logistic regression probabilistic model

$$\Pr\left(y_t \mid \mathbf{x}_t, \boldsymbol{\theta}, \theta_0\right) = g\left(y_t\left(\langle \boldsymbol{\theta}, \mathbf{x}_t\rangle + \theta_0\right)\right) = \frac{1}{1+\exp\left(-y_t\left(\langle \boldsymbol{\theta}, \mathbf{x}_t\rangle + \theta_0\right)\right)}, \tag{1.3}$$

- Take the gradients of

$$\log\left[1+\exp(-y_t(\langle \mathbf{x}_t, \boldsymbol{\theta}\rangle + \theta_0))\right]$$

with respect to $\boldsymbol{\theta}$ and θ_0 separately and update parameters $\boldsymbol{\theta}$ and θ_0 ($\eta > 0$: small learning rate)

$$\boldsymbol{\theta} \longleftarrow \boldsymbol{\theta} + \eta \cdot y_t \mathbf{x}_t\left[1 - \Pr(y_t \mid \mathbf{x}_t, \boldsymbol{\theta}, \theta_0)\right] \tag{1.24}$$

$$\theta_0 \longleftarrow \theta_0 + \eta \cdot y_t\left[1 - \Pr(y_t \mid \mathbf{x}_t, \boldsymbol{\theta}, \theta_0)\right]. \tag{1.25}$$

- Stochastic gradient descent: update $\boldsymbol{\theta}$ and θ_0 until the termination condition is met.



Summary of logistic regression: regularization

- The likelihood is strictly increasing as a function of $y_t(\langle \boldsymbol{\theta}^*, \mathbf{x}_t \rangle + \theta_0^*)$.

$$L(\boldsymbol{\theta}, \theta_0 \mid \mathcal{D}) = \prod_{t=1}^{n} \Pr(y_t \mid \mathbf{x}_t, \boldsymbol{\theta}, \theta_0) = \prod_{t=1}^{n} \frac{1}{1 + \exp\left(-y_t\left(\langle \boldsymbol{\theta}, \mathbf{x}_t \rangle + \theta_0\right)\right)}$$

- Infinite scaling of any parameters corresponding to a perfect linear classifier would attain the highest likelihood.
  - If the training data are linearly- or affinely-separable, we can find parameters $(\boldsymbol{\theta}^*, \theta_0^*)$ such that $y_t(\langle \boldsymbol{\theta}^*, \mathbf{x}_t \rangle + \theta_0^*) \geq 0$ for all t. By scaling up the parameters, we make these values larger and larger.
- Regularization: e.g., ℓ_2-norm

$$\min_{(\boldsymbol{\theta}, \theta_0) \in \mathbb{R}^d \times \mathbb{R}} \frac{\lambda}{2} \|\boldsymbol{\theta}\|^2 + \sum_{t=1}^{n} \log\left[1 + \exp(-y_t(\langle \mathbf{x}_t, \boldsymbol{\theta} \rangle + \theta_0))\right]. \tag{1.26}$$



Summary of logistic regression: multi-class classification

⊙ When we have $M \geq 2$ classes, replace

$$\Pr(y = 1 \mid \mathbf{x}, \boldsymbol{\theta}, \theta_0) = \frac{1}{1 + \exp\left(-\left(\langle \boldsymbol{\theta}, \mathbf{x} \rangle + \theta_0\right)\right)} \tag{1.2}$$

by the soft-max function

$$\Pr(y = c \mid \mathbf{x}, \{\boldsymbol{\theta}_c, \theta_{0,c}\}_{c=1}^{M}) = \frac{\exp(\langle \boldsymbol{\theta}_c, \mathbf{x} \rangle + \theta_{0,c})}{\sum_{c'=1}^{M} \exp(\langle \boldsymbol{\theta}_{c'}, \mathbf{x} \rangle + \theta_{0,c'})} \tag{1.27}$$

where we now have a different pair $(\boldsymbol{\theta}_c, \theta_{0,c})$ for each class.

• Without loss of generality, we can assume that one of the classes (say the first one) has $(\boldsymbol{\theta}_1, \theta_{0,1}) = (\mathbf{0}, 0)$, which is useful for showing that (1.2) is a special case of (1.27).
  ◦ **Why?**



Summary of logistic regression: further exploration

- See the impact of regularization parameter λ
- Try it for multi-class classification

- Find some dataset from Kaggle:
  `https://www.kaggle.com/code/rtatman/datasets-for-regression-analysis`
- Useful Python packages and functions
  `sklearn.linear_model.LogisticRegression`
  `numpy.random`
  `matplotlib.pyplot.scatter`



Outline

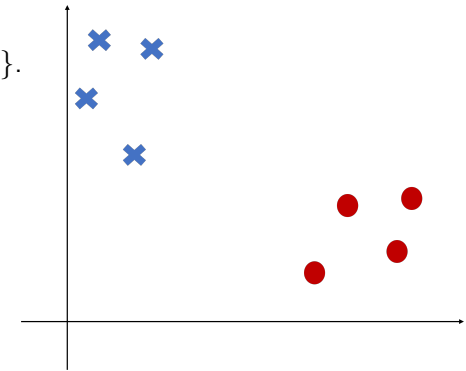
1 Logistic regression

2 Support vector machines (SVM)



Classification

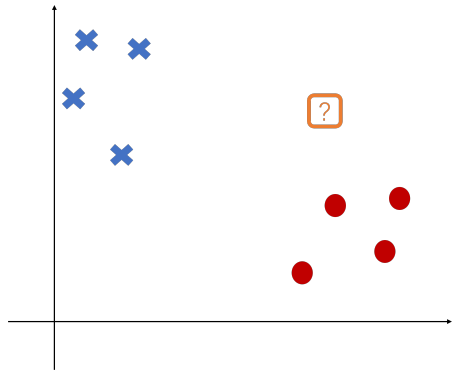
- Given a set of data
  $\mathcal{D} = \{(\mathbf{x}_t, y_t) \in \mathbb{R}^d \times \{-1, +1\} : t = 1, 2, \ldots, n\}$.





Classification

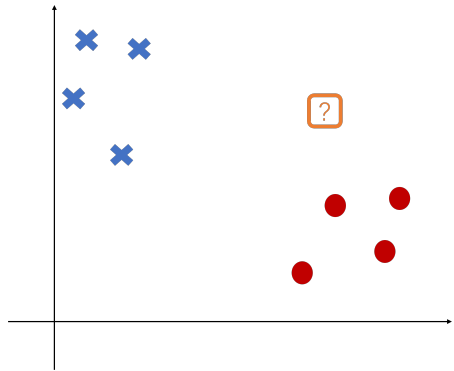
- Given a set of data
  $\mathcal{D} = \{(\mathbf{x}_t, y_t) \in \mathbb{R}^d \times \{-1, +1\} : t = 1, 2, \ldots, n\}$.
- How can we classify a new sample?
  i.e., what is the label y_{new} for x_{new}?





Classification

- Given a set of data
  $\mathcal{D} = \{(\mathbf{x}_t, y_t) \in \mathbb{R}^d \times \{-1, +1\} : t = 1, 2, \ldots, n\}$.
- How can we classify a new sample?
  i.e., what is the label y_{new} for x_{new}?

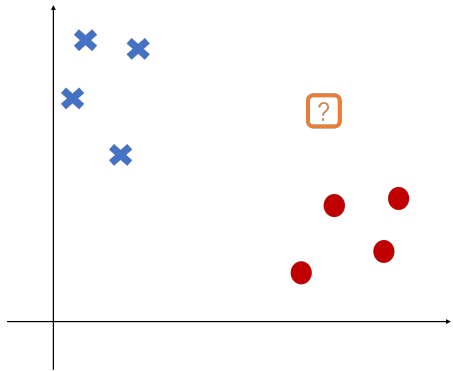


- Classifier Learnt: Linear classification ('hard'), logistic regression ('soft')



Classification

- Given a set of data
  $\mathcal{D} = \{(\mathbf{x}_t, y_t) \in \mathbb{R}^d \times \{-1, +1\} : t = 1, 2, \ldots, n\}$.
- How can we classify a new sample?
  i.e., what is the label y_{new} for x_{new}?



- Classifier Learnt: Linear classification ('hard'), logistic regression ('soft')
- **New classifier: Support Vector Machine (SVM)**



Margin of classifier

- Given a set of data $\mathcal{D} = \{(\mathbf{x}_t, y_t) \in \mathbb{R}^d \times \{-1, +1\}$, we can find some classifier.
- For any $\mathbf{x}, y$, correct classification by classifier $f_{\boldsymbol{\theta}} = \operatorname{sign}(\boldsymbol{\theta}^\top \mathbf{x})$ means

$$\begin{cases} \boldsymbol{\theta}^\top \mathbf{x} > 0, y = +1 \\ \boldsymbol{\theta}^\top \mathbf{x} < 0, y = -1 \end{cases} \Leftrightarrow \; y = \operatorname{sign}(\boldsymbol{\theta}^\top \mathbf{x}) \; \Leftrightarrow \; y\boldsymbol{\theta}^\top \mathbf{x} > 0.$$



Margin of classifier

- Given a set of data $\mathcal{D} = \{(\mathbf{x}_t, y_t) \in \mathbb{R}^d \times \{-1, +1\}$, we can find some classifier.
- For any $\mathbf{x}, y$, correct classification by classifier $f_{\boldsymbol{\theta}} = \operatorname{sign}(\boldsymbol{\theta}^\top \mathbf{x})$ means

$$\begin{cases} \boldsymbol{\theta}^\top \mathbf{x} > 0, y = +1 \\ \boldsymbol{\theta}^\top \mathbf{x} < 0, y = -1 \end{cases} \Leftrightarrow \; y = \operatorname{sign}(\boldsymbol{\theta}^\top \mathbf{x}) \; \Leftrightarrow \; y\boldsymbol{\theta}^\top \mathbf{x} > 0.$$

Definition 2.1 (Margin of classifier)

The **margin** of classifier $f_{\boldsymbol{\theta}} = \operatorname{sign}(\boldsymbol{\theta}^\top \mathbf{x})$ on sample $(\mathbf{x}, y)$ is

$$y\boldsymbol{\theta}^\top \mathbf{x} \text{ or } y\langle \boldsymbol{\theta}, \mathbf{x} \rangle$$

.



Margin of classifier

- Given a set of data $\mathcal{D} = \{(\mathbf{x}_t, y_t) \in \mathbb{R}^d \times \{-1, +1\}$, we can find some classifier.
- For any $\mathbf{x}, y$, correct classification by classifier $f_{\boldsymbol{\theta}} = \operatorname{sign}(\boldsymbol{\theta}^\top \mathbf{x})$ means

$$\begin{cases} \boldsymbol{\theta}^\top \mathbf{x} > 0, y = +1 \\ \boldsymbol{\theta}^\top \mathbf{x} < 0, y = -1 \end{cases} \Leftrightarrow \; y = \operatorname{sign}(\boldsymbol{\theta}^\top \mathbf{x}) \; \Leftrightarrow \; y\boldsymbol{\theta}^\top \mathbf{x} > 0.$$

Definition 2.1 (Margin of classifier)

The **margin** of classifier $f_{\boldsymbol{\theta}} = \operatorname{sign}(\boldsymbol{\theta}^\top \mathbf{x})$ on sample $(\mathbf{x}, y)$ is

$$y\boldsymbol{\theta}^\top \mathbf{x} \text{ or } y\langle \boldsymbol{\theta}, \mathbf{x} \rangle$$

.

- Positive margin: $(\mathbf{x}, y)$ is correctly classified by $\boldsymbol{\theta}$.
- Negative margin: $(\mathbf{x}, y)$ is **not** correctly classified by $\boldsymbol{\theta}$.



Margin of classifier

- Given a set of data $\mathcal{D} = \{(\mathbf{x}_t, y_t) \in \mathbb{R}^d \times \{-1, +1\}$, we can find some classifier.
- For any $\mathbf{x}, y$, correct classification by classifier $f_{\boldsymbol{\theta}} = \operatorname{sign}(\boldsymbol{\theta}^\top \mathbf{x})$ means

$$\begin{cases} \boldsymbol{\theta}^\top \mathbf{x} > 0, y = +1 \\ \boldsymbol{\theta}^\top \mathbf{x} < 0, y = -1 \end{cases} \Leftrightarrow \; y = \operatorname{sign}(\boldsymbol{\theta}^\top \mathbf{x}) \; \Leftrightarrow \; y\boldsymbol{\theta}^\top \mathbf{x} > 0.$$

Definition 2.1 (Margin of classifier)

The **margin** of classifier $f_{\boldsymbol{\theta}} = \operatorname{sign}(\boldsymbol{\theta}^\top \mathbf{x})$ on sample $(\mathbf{x}, y)$ is

$$y\boldsymbol{\theta}^\top \mathbf{x} \text{ or } y\langle \boldsymbol{\theta}, \mathbf{x} \rangle$$

.

- Positive margin: $(\mathbf{x}, y)$ is correctly classified by $\boldsymbol{\theta}$.
- Negative margin: $(\mathbf{x}, y)$ is **not** correctly classified by $\boldsymbol{\theta}$.
- Bigger margin: $(\mathbf{x}, y)$ is **more** correctly classified by $\boldsymbol{\theta}$.



Linearly separable

Definition 2.2 (Linearly separable)

The dataset $\mathcal{D} = \{(\mathbf{x}_t, y_t) \in \mathbb{R}^d \times \{-1, +1\} : t = 1, 2, \ldots, n\}$ is **linearly separable** if there exists some $\boldsymbol{\theta}$ such that

$$y_t = \operatorname{sign}(\boldsymbol{\theta}^\top \mathbf{x}_t) \text{ or } y_t \boldsymbol{\theta}^\top \mathbf{x}_t > 0 \qquad \forall\, t = 1, 2, \ldots, n. \tag{2.1}$$



Linearly separable

Definition 2.2 (Linearly separable)

The dataset $\mathcal{D} = \{(\mathbf{x}_t, y_t) \in \mathbb{R}^d \times \{-1, +1\} : t = 1, 2, \ldots, n\}$ is **linearly separable** if there exists some $\boldsymbol{\theta}$ such that

$$y_t = \operatorname{sign}(\boldsymbol{\theta}^\top \mathbf{x}_t) \text{ or } y_t \boldsymbol{\theta}^\top \mathbf{x}_t > 0 \qquad \forall\, t = 1, 2, \ldots, n. \tag{2.1}$$

There is some hyperplane that (strictly) separates the data into positive and negative samples



Linearly separable

Definition 2.2 (Linearly separable)

The dataset $\mathcal{D} = \{(\mathbf{x}_t, y_t) \in \mathbb{R}^d \times \{-1, +1\} : t = 1, 2, \ldots, n\}$ is **linearly separable** if there exists some $\boldsymbol{\theta}$ such that

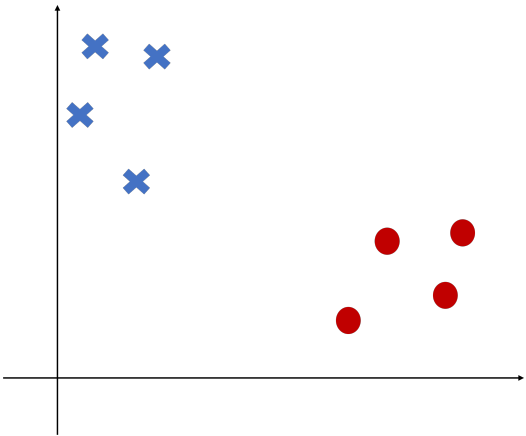
$$y_t = \operatorname{sign}(\boldsymbol{\theta}^\top \mathbf{x}_t) \text{ or } y_t \boldsymbol{\theta}^\top \mathbf{x}_t > 0 \qquad \forall\, t = 1, 2, \ldots, n. \tag{2.1}$$

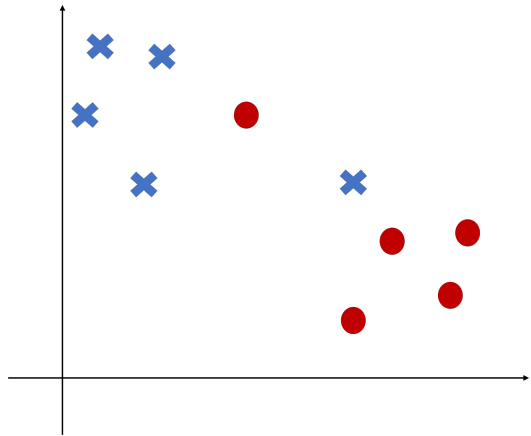
There is some hyperplane that (strictly) separates the data into positive and negative samples

- $\boldsymbol{\theta}$ has positive margin $y_t \langle \boldsymbol{\theta}, \mathbf{x}_t \rangle > 0$ for all t
- **Minimum margin** $\gamma = \min\limits_{1 \le t \le n} y_t \langle \boldsymbol{\theta}, \mathbf{x}_t \rangle > 0$

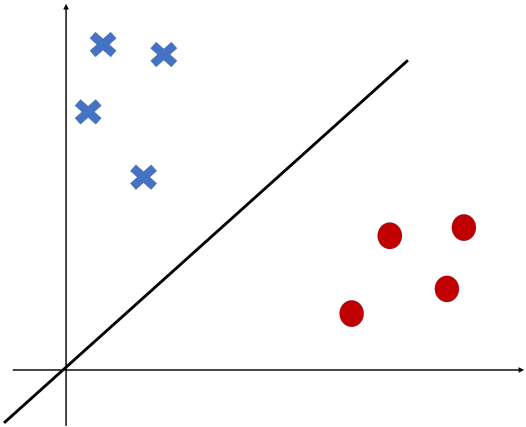


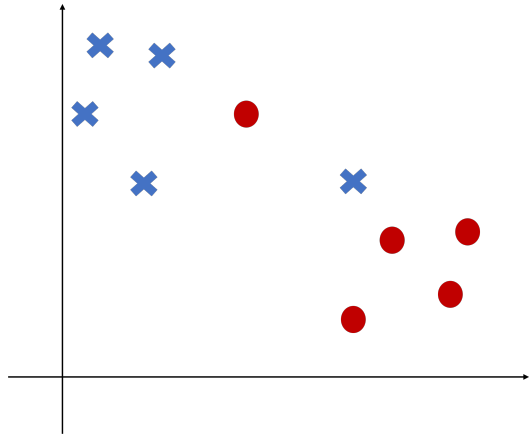
Linearly separable





Linearly separable





γ-linearly separable

Definition 2.3 (γ-linearly separable)

The dataset $\mathcal{D} = \{(\mathbf{x}_t, y_t) \in \mathbb{R}^d \times \{-1, +1\} : t = 1, 2, \ldots, n\}$ is γ-**linearly separable** for some $\gamma > 0$ if there exists some $\boldsymbol{\theta}$ such that

$$y_t \langle \boldsymbol{\theta}, \mathbf{x}_t \rangle \geq \gamma \qquad \forall\, t = 1, 2, \ldots, n. \tag{2.2}$$



γ-linearly separable

Definition 2.3 (γ-linearly separable)

The dataset $\mathcal{D} = \{(\mathbf{x}_t, y_t) \in \mathbb{R}^d \times \{-1, +1\} : t = 1, 2, \ldots, n\}$ is γ-**linearly separable** for some $\gamma > 0$ if there exists some $\boldsymbol{\theta}$ such that

$$y_t \langle \boldsymbol{\theta}, \mathbf{x}_t \rangle \geq \gamma \qquad \forall\, t = 1, 2, \ldots, n. \tag{2.2}$$

There is some hyperplane that (strictly) separates the data into positive and negative samples

- $\boldsymbol{\theta}$ has positive margin $y_t \langle \boldsymbol{\theta}, \mathbf{x}_t \rangle > 0$ for all t
- **Minimum margin** $\gamma = \min_{1 \leq t \leq n} y_t \langle \boldsymbol{\theta}, \mathbf{x}_t \rangle > 0$

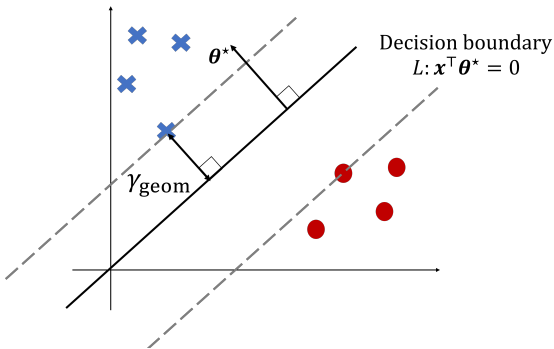


Geometric margin

⊙ **Geometric margin:**

$$\gamma_{\text{geom}} := \frac{\gamma}{\|\boldsymbol{\theta}\|} = \frac{\min\limits_{1\le t\le n} y_t \mathbf{x}_t^\top \boldsymbol{\theta}}{\|\boldsymbol{\theta}\|}.$$

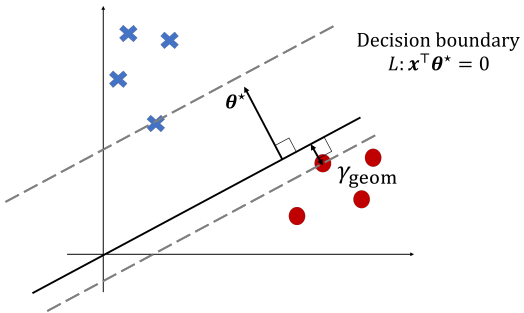
- Smallest distance over all samples to the decision boundary $L = \{x \in \mathbb{R}^d : \langle \mathbf{x}, \boldsymbol{\theta} \rangle = 0\}$.
- $\gamma_{\text{geom}}^{-1}$: fair measure of the difficulty of the classification problem

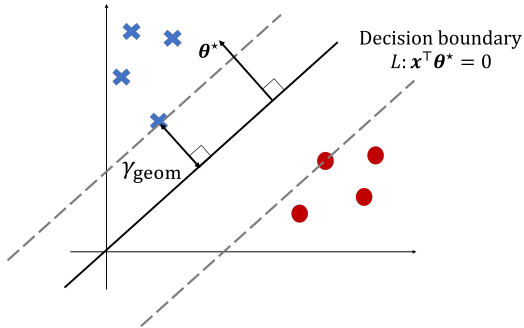




Maximum margin linear classifier

Q: Why do we want to maximize the geometric margin?

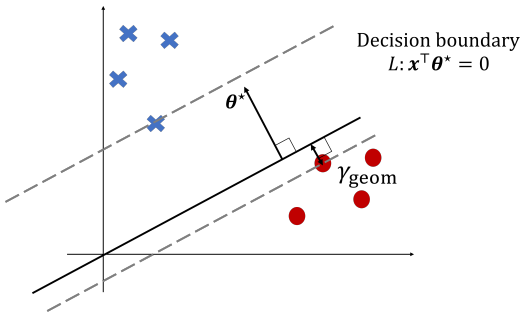


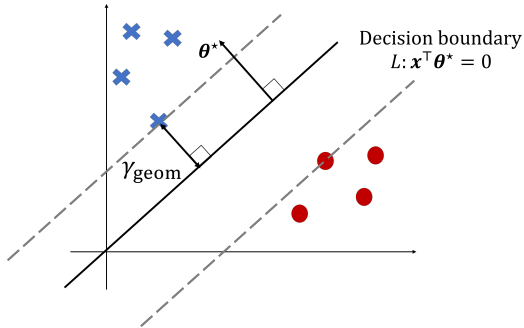


Maximum margin linear classifier

Q: Why do we want to maximize the geometric margin?

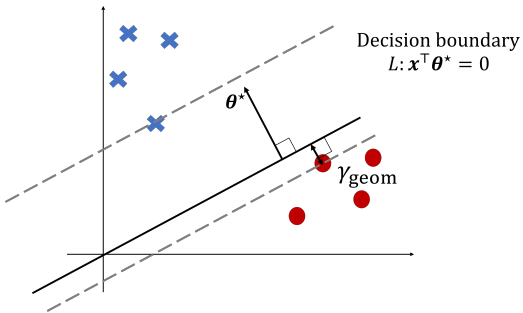
A: Larger geometric margin: more robust to noisy samples...





Maximum margin linear classifier

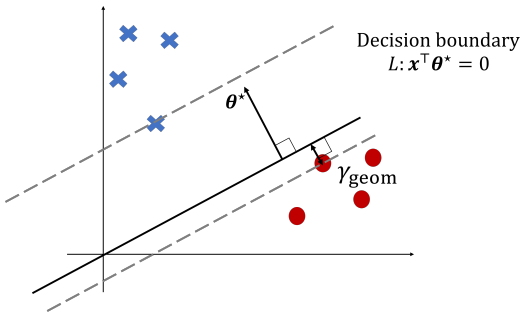
- Can we maximize this geometric margin directly?

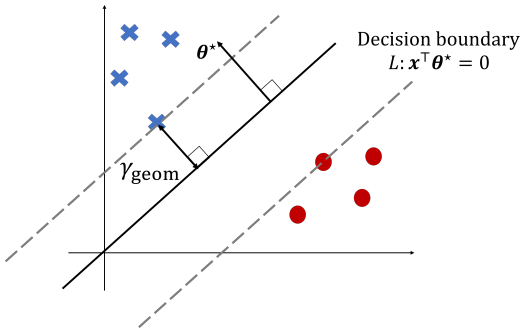




Maximum margin linear classifier

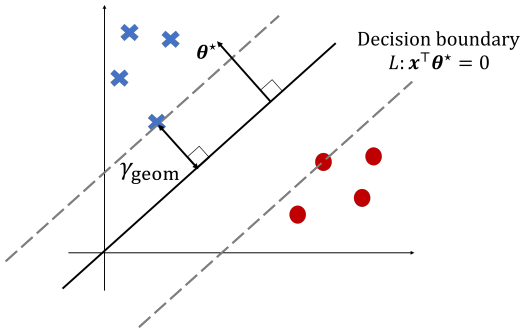
- Can we maximize this geometric margin directly?

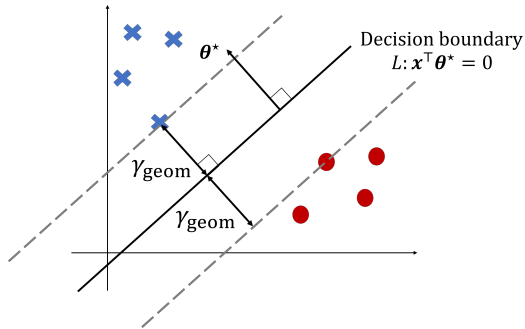




Maximum margin linear classifier

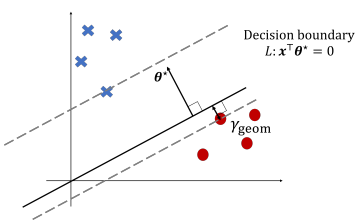
- Can we maximize this geometric margin directly?

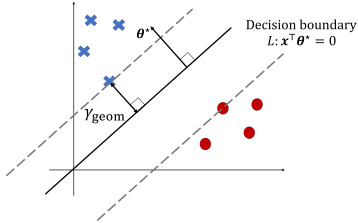


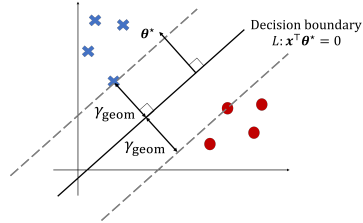


Maximum margin linear classifier

- Can we maximize this geometric margin directly?
- In principle, we can
  1. Find a classifier that correctly classifies all the samples
  2. Increase the geometric margin until the classifier 'locks in place'









Maximum margin linear classifier

- We can prove that the solution is unique.
- For γ-linearly separable dataset
  - Optimization problem:

$$\max_{\boldsymbol{\theta}' \in \mathbb{R}^d} \frac{\gamma}{\|\boldsymbol{\theta}'\|} \text{ s. t. } y_t \mathbf{x}_t^\top \boldsymbol{\theta}' \geq \gamma, \quad \forall t = 1, 2, \ldots, n$$

  - Maximization $\longrightarrow$ minimization:

$$\min_{\boldsymbol{\theta}' \in \mathbb{R}^d} \frac{\|\boldsymbol{\theta}'\|}{\gamma} \text{ s. t. } y_t \mathbf{x}_t^\top \boldsymbol{\theta}' \geq \gamma, \quad \forall t = 1, 2, \ldots, n$$



Maixmum margin linear classifier

- Since $\gamma > 0$, this can be rewritten as:

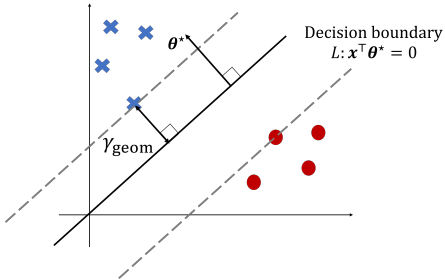
$$\min_{\boldsymbol{\theta}' \in \mathbb{R}^d} \left\| \frac{\boldsymbol{\theta}'}{\gamma} \right\| \text{ s. t. } y_t \mathbf{x}_t^\top \frac{\boldsymbol{\theta}'}{\gamma} \geq 1, \quad \forall t = 1, 2, \ldots, n$$

- The classification problem is dependent on the ratio $\frac{\boldsymbol{\theta}'}{\gamma}$, not $\boldsymbol{\theta}'$ or γ separately.
  e.g. Geometric margin $\gamma_{\text{geom}} = \frac{\gamma}{\|\boldsymbol{\theta}'\|}$.
- $\approx$ Optimize $\boldsymbol{\theta}'$ and γ together by optimizing $\frac{\boldsymbol{\theta}'}{\gamma}$



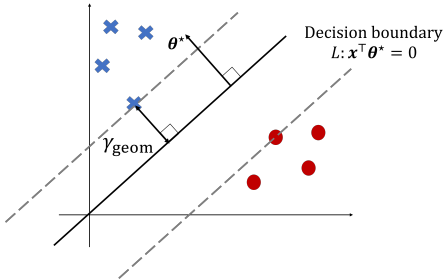
Maximum margin linear classifier

- Scaling $\boldsymbol{\theta}'$ by a constant does not change the decision boundary



Maximum margin linear classifier

- Scaling $\boldsymbol{\theta}'$ by a constant does not change the decision boundary



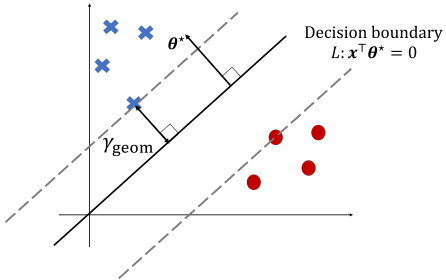
- Let $\boldsymbol{\theta}' \longleftarrow \frac{\boldsymbol{\theta}'}{\gamma}$ and solve for $\boldsymbol{\theta}'$ from

$$\min_{\boldsymbol{\theta}' \in \mathbb{R}^d} \left\|\boldsymbol{\theta}'\right\| \text{ s. t. } y_t \mathbf{x}_t^{\top} \boldsymbol{\theta}' \geq 1, \quad \forall t = 1, 2, \ldots, n$$



Maximum margin linear classifier

- Scaling $\boldsymbol{\theta}'$ by a constant does not change the decision boundary



- Let $\boldsymbol{\theta}' \longleftarrow \frac{\boldsymbol{\theta}'}{\gamma}$ and solve for $\boldsymbol{\theta}'$ from

$$\min_{\boldsymbol{\theta}' \in \mathbb{R}^d} \left\|\boldsymbol{\theta}'\right\| \text{ s. t. } y_t \mathbf{x}_t^{\top} \boldsymbol{\theta}' \geq 1, \quad \forall t = 1, 2, \ldots, n$$

$\Longleftrightarrow$ **Primal form of SVM:**

$$\text{(PRIMAL-SVM)} \quad \min_{\boldsymbol{\theta}' \in \mathbb{R}^d} \frac{1}{2} \left\|\boldsymbol{\theta}'\right\|^2 \text{ s. t. } y_t \mathbf{x}_t^{\top} \boldsymbol{\theta}' \geq 1, \quad \forall t = 1, 2, \ldots, n$$



Support Vector Machine (SVM)

- Primal form of SVM:

$$(\text{PRIMAL-SVM}) \quad \min_{\boldsymbol{\theta}' \in \mathbb{R}^d} \frac{1}{2} \left\|\boldsymbol{\theta}'\right\|^2 \text{ s. t. } y_t \mathbf{x}_t^\top \boldsymbol{\theta}' \geq 1, \quad \forall t = 1, 2, \ldots, n$$

- SVM algorithm: find the maximum (geometric) margin linear classifier

"the most theoretically well motivated and practically most effective classification algorithms in modern machine learning"



Primal-SVM: unique solution



Primal-SVM: unique solution

Proof:

- Suppose to the contrary, $\exists 2$ distinct min length solution vectors $\boldsymbol{\theta}_1, \boldsymbol{\theta}_2$ $(\boldsymbol{\theta}_1 \neq \boldsymbol{\theta}_2)$



Primal-SVM: unique solution

Proof:

- Suppose to the contrary, $\exists 2$ distinct min length solution vectors $\boldsymbol{\theta}_1, \boldsymbol{\theta}_2$ $(\boldsymbol{\theta}_1 \neq \boldsymbol{\theta}_2)$
- Then $\|\boldsymbol{\theta}_1\| = \|\boldsymbol{\theta}_2\|$ (otherwise they cannot both be the min length solutions)



Primal-SVM: unique solution

Proof:

- Suppose to the contrary, $\exists 2$ distinct min length solution vectors $\boldsymbol{\theta}_1, \boldsymbol{\theta}_2$ $(\boldsymbol{\theta}_1 \neq \boldsymbol{\theta}_2)$
- Then $\|\boldsymbol{\theta}_1\| = \|\boldsymbol{\theta}_2\|$ (otherwise they cannot both be the min length solutions)
- Let $\bar{\boldsymbol{\theta}} = (\boldsymbol{\theta}_1 + \boldsymbol{\theta}_2)/2$



Primal-SVM: unique solution

Proof:

- Suppose to the contrary, $\exists 2$ distinct min length solution vectors $\boldsymbol{\theta}_1, \boldsymbol{\theta}_2$ $(\boldsymbol{\theta}_1 \neq \boldsymbol{\theta}_2)$
- Then $\|\boldsymbol{\theta}_1\| = \|\boldsymbol{\theta}_2\|$ (otherwise they cannot both be the min length solutions)
- Let $\bar{\boldsymbol{\theta}} = (\boldsymbol{\theta}_1 + \boldsymbol{\theta}_2)/2$
- Check feasibility:



Primal-SVM: unique solution

Proof:

- Suppose to the contrary, $\exists 2$ distinct min length solution vectors $\boldsymbol{\theta}_1, \boldsymbol{\theta}_2$ $(\boldsymbol{\theta}_1 \neq \boldsymbol{\theta}_2)$
- Then $\|\boldsymbol{\theta}_1\| = \|\boldsymbol{\theta}_2\|$ (otherwise they cannot both be the min length solutions)
- Let $\bar{\boldsymbol{\theta}} = (\boldsymbol{\theta}_1 + \boldsymbol{\theta}_2)/2$
- Check feasibility:

  Since $\forall t = 1, 2, \ldots, n$, $y_t \mathbf{x}_t^\top \boldsymbol{\theta}_1 \geq 1$ and $y_t \mathbf{x}_t^\top \boldsymbol{\theta}_2 \geq 1$, by linearity of inner product, we have

$$y_t \mathbf{x}_t^\top \bar{\boldsymbol{\theta}} = \frac{1}{2} y_t \mathbf{x}_t^\top \boldsymbol{\theta}_1 + \frac{1}{2} y_t \mathbf{x}_t^\top \boldsymbol{\theta}_2 \geq 1$$



Primal-SVM: unique solution

Proof:

- Suppose to the contrary, $\exists 2$ distinct min length solution vectors $\boldsymbol{\theta}_1, \boldsymbol{\theta}_2$ $(\boldsymbol{\theta}_1 \neq \boldsymbol{\theta}_2)$
- Then $\|\boldsymbol{\theta}_1\| = \|\boldsymbol{\theta}_2\|$ (otherwise they cannot both be the min length solutions)
- Let $\bar{\boldsymbol{\theta}} = (\boldsymbol{\theta}_1 + \boldsymbol{\theta}_2)/2$
- Check feasibility:

  Since $\forall t = 1, 2, \ldots, n$, $y_t \mathbf{x}_t^\top \boldsymbol{\theta}_1 \geq 1$ and $y_t \mathbf{x}_t^\top \boldsymbol{\theta}_2 \geq 1$, by linearity of inner product, we have

$$y_t \mathbf{x}_t^\top \bar{\boldsymbol{\theta}} = \frac{1}{2} y_t \mathbf{x}_t^\top \boldsymbol{\theta}_1 + \frac{1}{2} y_t \mathbf{x}_t^\top \boldsymbol{\theta}_2 \geq 1$$

  $\Rightarrow \bar{\boldsymbol{\theta}}$ is feasible



Primal-SVM: unique solution

Proof (continued):

$$\left\|\bar{\boldsymbol{\theta}}\right\| = \left\|\frac{1}{2}\left(\boldsymbol{\theta}_1 + \boldsymbol{\theta}_2\right)\right\| \overset{\text{Triangle inequality}}{\leq} \frac{1}{2}\left\|\boldsymbol{\theta}_1\right\| + \frac{1}{2}\left\|\boldsymbol{\theta}_2\right\|$$

$$= \left\|\boldsymbol{\theta}_1\right\| = \left\|\boldsymbol{\theta}_2\right\|$$

Two case:

1. $\|\bar{\boldsymbol{\theta}}\| < \|\boldsymbol{\theta}_1\| = \|\boldsymbol{\theta}_2\| \Rightarrow$ Contradiction!
2. $\|\bar{\boldsymbol{\theta}}\| = \|\boldsymbol{\theta}_1\| = \|\boldsymbol{\theta}_2\| \Rightarrow$ Triangle inequality holds with equality. This can only happen if $\boldsymbol{\theta}_1$ is in the same direction as $\boldsymbol{\theta}_2$.

In conclusion: there DO NOT exist 2 distinct min length solution vectors to PRIMAL-SVM.



Support Vector Machine (SVM): unique solution to (Primal-SVM)

- Primal form of SVM:

$$(\text{PRIMAL-SVM}) \quad \min_{\boldsymbol{\theta}' \in \mathbb{R}^d} \frac{1}{2} \left\|\boldsymbol{\theta}'\right\|^2 \text{ s. t. } y_t \mathbf{x}_t^\top \boldsymbol{\theta}' \geq 1, \quad \forall t = 1, 2, \ldots, n$$

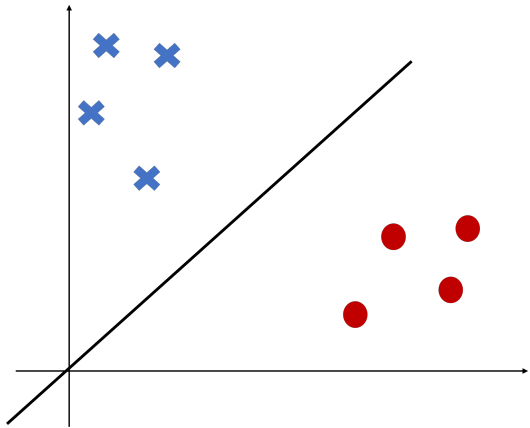
- SVM algorithm: can find the unique maximum (geometric) margin linear classifier by solving (PRIMAL-SVM).

"the most theoretically well motivated and practically most effective classification algorithms in modern machine learning"



SVM without offset

- Till now, we only considered the linear classifier without offset: $f_{\boldsymbol{\theta}}(\mathbf{x}) = \operatorname{sign}(\boldsymbol{\theta}^\top \mathbf{x})$

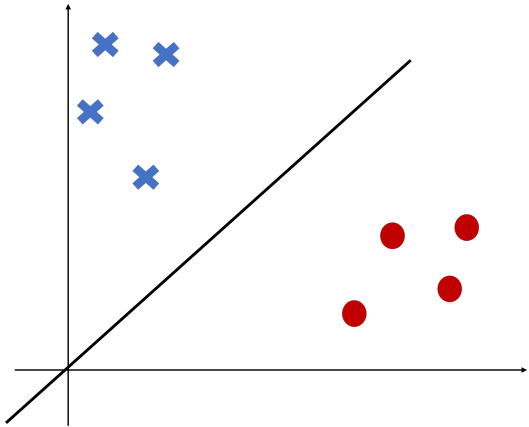




SVM without offset

- Till now, we only considered the linear classifier without offset: $f_{\boldsymbol{\theta}}(\mathbf{x}) = \operatorname{sign}(\boldsymbol{\theta}^\top \mathbf{x})$

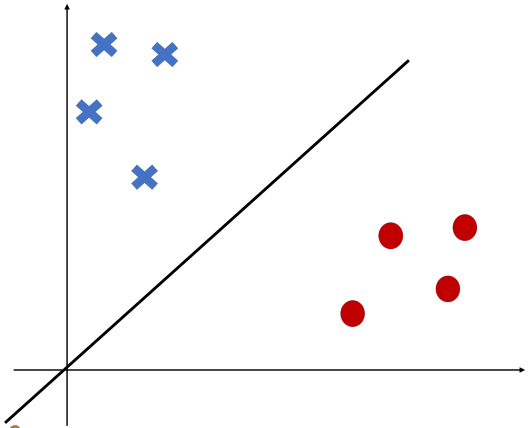
⊙ **Q:** How can we further maximize the geometric margin γ_{geom}?

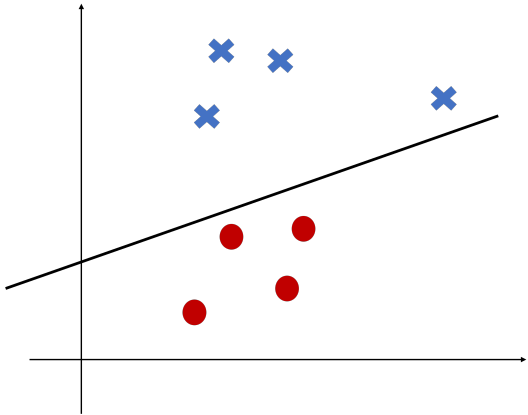


Incorporating an offset

- Till now, we only considered the linear classifier without offset: $f_{\boldsymbol{\theta}}(\mathbf{x}) = \text{sign}(\boldsymbol{\theta}^\top \mathbf{x})$

⊙ **Q:** How can we further maximize the geometric margin γ_{geom}?





Incorporating an offset

- Till now, we only considered the linear classifier without offset: $f_{\boldsymbol{\theta}}(\mathbf{x}) = \operatorname{sign}(\boldsymbol{\theta}^\top \mathbf{x})$
- If data are not "centered" at zero, we need to incorporate an offset $\theta_0 \in \mathbb{R}$:

$$f_{\boldsymbol{\theta},b}(\mathbf{x}) = \operatorname{sign}(\boldsymbol{\theta}^\top \mathbf{x} + \theta) = \operatorname{sign}\left(\begin{bmatrix}\boldsymbol{\theta}\\ \theta_0\end{bmatrix}^\top \begin{bmatrix}\mathbf{x}\\ 1\end{bmatrix}\right)$$

- Then the decision boundary is

$$L = \{x \in \mathbb{R}^d : \boldsymbol{\theta}^\top \mathbf{x} + \theta_0 = 0\}$$

If $\theta = 0$, the line L passes through the origin.



Incorporating an offset

- How to find the maximum margin classifier with an offset?
- Optimization problem: (PRIMAL-SVM WITH OFFSET)

$$\min_{(\boldsymbol{\theta},\theta)\in\mathbb{R}^d\times\mathbb{R}} \frac{1}{2}\|\boldsymbol{\theta}\|^2 \text{ s. t. } y_t(\mathbf{x}_t^\top \boldsymbol{\theta} + \theta_0) \geq 1, \quad \forall t = 1, 2, \ldots, n$$

Remarks:

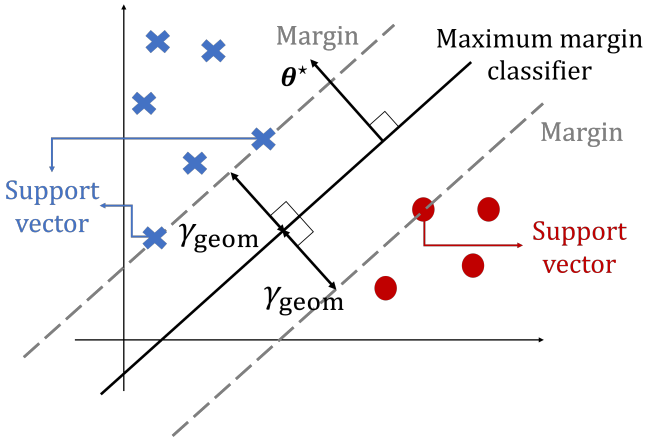
1. The offset b only appears in constraints
2. Different from $\tilde{\mathbf{x}} = [\mathbf{x}; 1]$:

   $\longrightarrow$ We do not bias in any way where the separating hyperplane should appear, only that it should maximize the geometric margin.



Support Vector + Machine

- Support vector $\Leftarrow$ come from the input data set
  influence the position of the max-margin classifier
- Machine: Input $\Longrightarrow$ Machine $\Longrightarrow$ Output classifier



Robustness of max margin classifier

- As we have mentioned: larger margin $\Rightarrow$ more robust

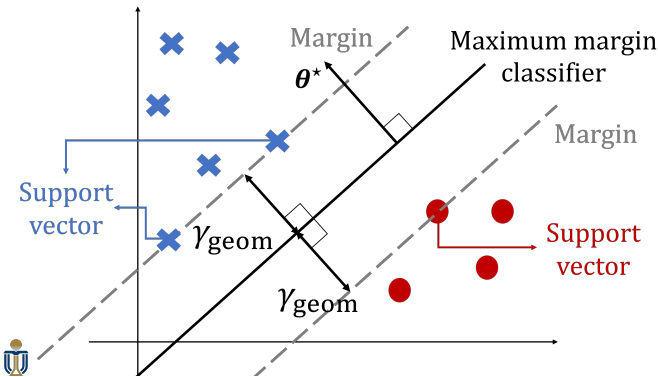


Robustness of max margin classifier

- As we have mentioned: larger margin $\Rightarrow$ more robust
- The (Primal-SVM with Offset) problem:

$$\min_{(\boldsymbol{\theta},\theta)\in\mathbb{R}^d\times\mathbb{R}} \frac{1}{2}\|\boldsymbol{\theta}\|^2 \text{ s. t. } y_t(\mathbf{x}_t^\top\boldsymbol{\theta}+\theta_0)\geq 1, \quad \forall t=1,2,\ldots,n$$

- Solution depends on a small number of data samples $(\mathbf{x}_t, y_t)$



Support vectors:

- Samples exactly on the margin.
- Quantify the quality of this binary classifier.

SVM: evaluation

$$\frac{\#\text{ incorrect prediction}}{\#\text{ all prediction}} = \frac{\sum_{t=1}^{n} \mathrm{Loss}(y_t, f(\mathbf{x}_t, (\boldsymbol{\theta}, \theta_0)))}{n}$$
$$= \frac{\sum_{t=1}^{n} \mathbb{1}\{y_t, f(\mathbf{x}_t, (\boldsymbol{\theta}, \theta_0))\}}{n}?$$



Leave-one-out cross-validation (LOOCV) error

To assess the robustness of the SVM

$$\text{LOOCV} = \frac{1}{n}\sum_{t=1}^{n} \text{Loss}(y_t, f(\mathbf{x}_t, (\boldsymbol{\theta}^{-t}, \theta_0^{-t}))),$$

- $(\boldsymbol{\theta}^{-t}, \theta_0^{-t})$ is learnt from the dataset with i^{th} sample left out, i.e., $\mathcal{D}^{-t} = \mathcal{D} \setminus \{\mathbf{x}_t, y_t\}$
- Loss function: $\text{Loss} = \mathbb{1}\{y \neq y'\}$ is zero-one loss



Leave-one-out cross-validation (LOOCV) error

To assess the robustness of the SVM

$$\mathrm{LOOCV} = \frac{1}{n}\sum_{t=1}^{n} \mathrm{Loss}(y_t, f(\mathbf{x}_t, (\boldsymbol{\theta}^{-t}, \theta_0^{-t}))),$$

- $(\boldsymbol{\theta}^{-t}, \theta_0^{-t})$ is learnt from the dataset with i^{th} sample left out, i.e., $\mathcal{D}^{-t} = \mathcal{D} \setminus \{\mathbf{x}_t, y_t\}$
- Loss function: $\mathrm{Loss} = \mathbb{1}\{y \neq y'\}$ is zero-one loss

In other words,

$$(\boldsymbol{\theta}^{-t}, \theta_0^{-t}) \in \underset{(\boldsymbol{\theta},\theta_0)\in\mathbb{R}^d\times\mathbb{R}}{\arg\min} \left\{ \frac{1}{2}\|\boldsymbol{\theta}\|^2 : y_t(\mathbf{x}_t^\top \boldsymbol{\theta} + \theta_0) \geq 1, \quad \forall t \in [n] \setminus \{t\} \right\}.$$



Leave-one-out cross-validation (LOOCV) error

If LOOCV is small, the model generalizes well (fits well on new samples)



Leave-one-out cross-validation (LOOCV) error

If LOOCV is small, the model generalizes well (fits well on new samples)

Proposition 2.4

Let the number of **support vectors** *be* $N \in \{0, 1, \ldots, n\}$. *Then*

$$\mathrm{LOOCV} \leq \frac{N}{n}.$$



Leave-one-out cross-validation (LOOCV) error

If LOOCV is small, the model generalizes well (fits well on new samples)

Proposition 2.4

Let the number of **support vectors** *be* $N \in \{0, 1, \ldots, n\}$. *Then*

$$\text{LOOCV} \leq \frac{N}{n}.$$

- If the number of support vectors is small, then LOOCV is also small and the SVM works well on new samples



Leave-one-out cross-validation (LOOCV) error

Proof:

- If $\mathbf{x}_t$ is not a support vector, then it cannot influence the solution of the primal SVM.
- Hence,

$$f(\mathbf{x}_t, (\boldsymbol{\theta}^{-t}, \theta^{-t})) = y_t, \quad \forall \mathbf{x}_t \notin \text{Support Vectors (SV)}$$

and

$$\text{Loss}(y_t, f(\mathbf{x}_t, (\boldsymbol{\theta}^{-t}, \theta_0^{-t}))) = 0, \quad \forall \mathbf{x}_t \notin \text{Support Vectors (SV)}$$

- For any other $\mathbf{x}_t \in$ SV,

$$\text{Loss}(y_t, f(\mathbf{x}_t, (\boldsymbol{\theta}^{-t}, \theta_0^{-t}))) \leq 1, \quad \forall \mathbf{x}_t \in \text{Support Vectors (SV)}$$



Leave-one-out cross-validation (LOOCV) error

Proof (continued):

$$\begin{aligned}\mathrm{LOOCV} &= \frac{1}{n}\left(\sum_{t\notin \mathrm{SV}} + \sum_{t\in \mathrm{SV}}\right)\mathrm{Loss}(y_t, f(\mathbf{x}_t, (\boldsymbol{\theta}^{-t}, \theta_0^{-t})))\\ &\leq 0 + \frac{1}{n}\sum_{i\in \mathrm{SV}} 1 = \frac{N}{n}.\end{aligned}$$

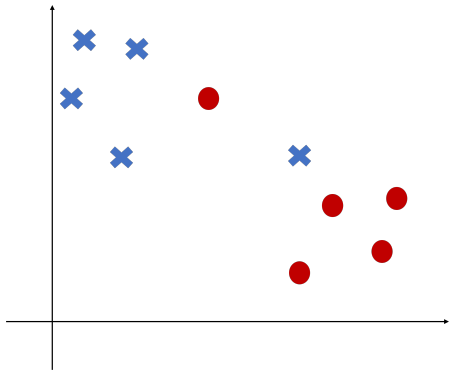
In conclusion: we use LOOCV to assess the robustness of the SVM.



Allowing misclassified example

To be more robust?

What if the data is not linear/affinely-separable?

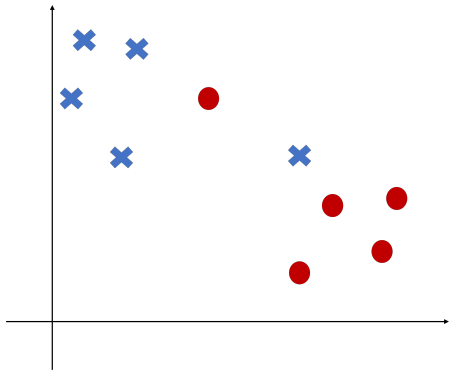


Allowing misclassified example

To be more robust?

What if the data is not linear/affinely-separable?

Simple way: permit some errors





Allowing misclassified example

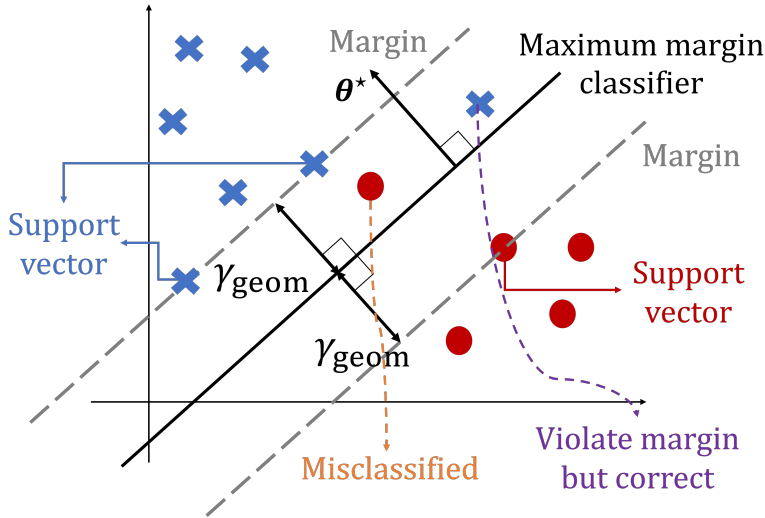
- The primal form of SVM problem with offset and slack:

$$\min_{\boldsymbol{\theta},\theta_0,\boldsymbol{\xi}\in\mathbb{R}^d\times\mathbb{R}\times\mathbb{R}^n_+} \frac{1}{2}\left\|\boldsymbol{\theta}'\right\|^2 + C\sum_{t=1}^{n}\xi_t$$
$$\text{s. t. } y_t(\mathbf{x}_t^\top\boldsymbol{\theta}+\theta_0) \geq 1-\xi_t,\ \xi_t \geq 0 \quad \forall t = 1,2,\ldots,n$$

where $\boldsymbol{\xi} = (\xi_1, \xi_2, \ldots, \xi_n)$ are slack variables and $C > 0$ is a pre-specified constant.



Allowing misclassified example



Allowing misclassified example

(Primal-SVM with offset and slack)

$$\min_{\boldsymbol{\theta},\theta_0,\boldsymbol{\xi}\in\mathbb{R}^d\times\mathbb{R}\times\mathbb{R}^n_+} \frac{1}{2}\left\|\boldsymbol{\theta}'\right\|^2 + C\sum_{t=1}^{n}\xi_t$$

$$\text{s. t. } y_t(\mathbf{x}_t^\top\boldsymbol{\theta}+\theta_0) \geq 1-\xi_t,\ \xi_t \geq 0 \quad \forall t = 1,2,\ldots,n$$

- The margin constraint is violated when we have to set $\xi_t > 0$ for some example $\mathbf{x}_t$.
- Penalty for this violation: $C\xi_t > 0$.



Allowing misclassified example

(Primal-SVM with offset and slack)

$$\min_{\boldsymbol{\theta},\theta_0,\boldsymbol{\xi}\in\mathbb{R}^d\times\mathbb{R}\times\mathbb{R}^n_+} \frac{1}{2}\left\|\boldsymbol{\theta}'\right\|^2 + C\sum_{t=1}^{n}\xi_t$$

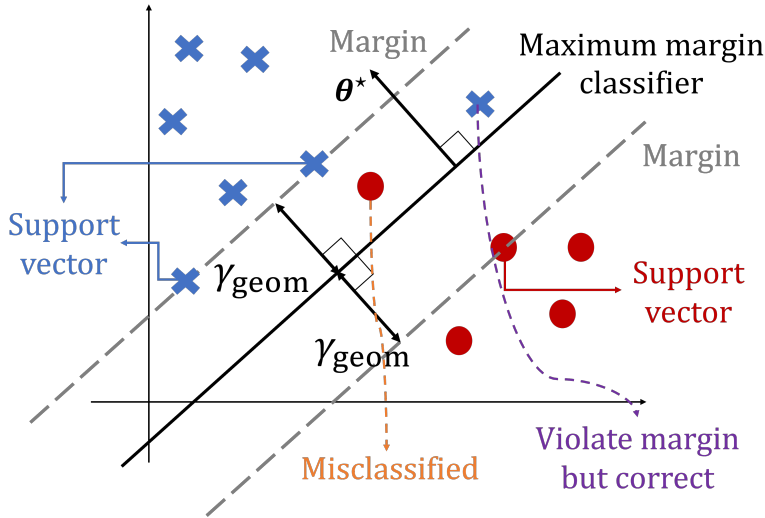
$$\text{s. t. } y_t(\mathbf{x}_t^\top\boldsymbol{\theta}+\theta_0) \geq 1-\xi_t,\ \xi_t \geq 0 \quad \forall t = 1,2,\ldots,n$$

- The margin constraint is violated when we have to set $\xi_t > 0$ for some example $\mathbf{x}_t$.
- Penalty for this violation: $C\xi_t > 0$.
- Trade-off between max margin objective $\|\boldsymbol{\theta}\|^2/2$ and margin violation $C\sum_{t=1}^{t}\xi_t$
- If $C \to \infty$, then $\xi_t \downarrow 0$ for all t

  $\Rightarrow$ the original max margin classifier without slack
- If $C \approx 0$, some margin violation is allowed



Allowing misclassified example



Allowing misclassified example

(Primal-SVM with offset and slack)

$$\min_{\boldsymbol{\theta},\theta_0,\boldsymbol{\xi}\in\mathbb{R}^d\times\mathbb{R}\times\mathbb{R}^n_+} \frac{1}{2}\left\|\boldsymbol{\theta}'\right\|^2 + C\sum_{t=1}^{n}\xi_t$$

$$\text{s. t. } y_t(\mathbf{x}_t^\top\boldsymbol{\theta}+\theta_0) \geq 1-\xi_t,\ \xi_t \geq 0 \quad \forall t = 1,2,\ldots,n$$

$$\Longleftrightarrow$$

$$\min_{\boldsymbol{\theta},\theta_0\in\mathbb{R}^d\times\mathbb{R}} \frac{1}{2}\left\|\boldsymbol{\theta}'\right\|^2 + C\sum_{t=1}^{n}\underbrace{\left[1-y_t(\mathbf{x}_t^\top\boldsymbol{\theta}+\theta_0)\right]^+}_{:=\xi_t}$$

where $f(z) = (1-z)^+ = \max\{0, 1-z\}$.



Support Vector Machine (SVM)

- An algorithm that learns the maximum (geometric) margin linear classifier
- The solution is unique
- Incorporating an offset may result in larger geometric margin
- LOOCV to assess the robustness of SVM
- For non-linearly separable data: allow misclassified examples by incorporating a slack



Support Vector Machine (SVM)

- An algorithm that learns the maximum (geometric) margin linear classifier
- The solution is unique
- Incorporating an offset may result in larger geometric margin
- LOOCV to assess the robustness of SVM
- For non-linearly separable data: allow misclassified examples by incorporating a slack

- Solving SVM: Dual form, sequential minimal optimization (SMO)...
- ⊙ Reading: Note of 6-867 (MIT)
  Sec 7.2 and Chapter 12 of book "Mathematics for Machine Learning"



Thanks for listening

1. Tell us your question/feedback via the QR code.
2. Lab reminder: Th 7-8PM[LA01] / 6-7PM[LA02]



⊙ Slides credit: some slides are adapted from (alphabetical order)
Dan Klein and Pieter Abbeel (UC Berkeley), Haiyun He (Cornell) and Tommi S. Jaakkola (MIT).

⊙ Book: 'Convex Optimization' by Stephen Boyd and Lieven Vandenberghe
'Multivariable Calculus' by James Stewart
'Pattern Recognition and Machine Learning' by Christopher M. Bishop

